

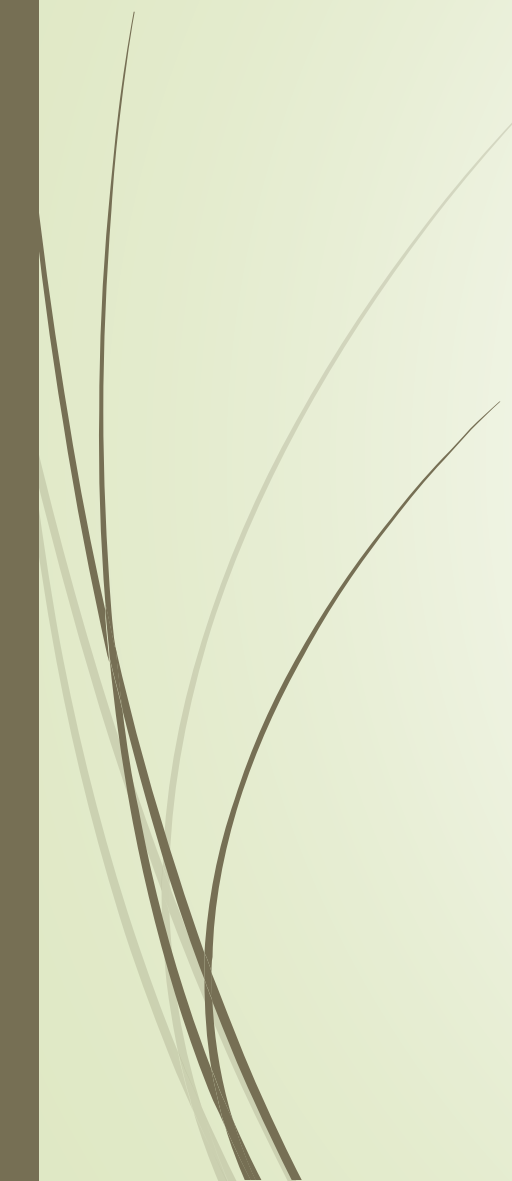


Image Processing

Chapter 3



Contents



- ★ Point operators
- ★ Linear filtering
- ★ OpenCV set up and reading assignment for OpenCV
- ★ More neighborhood operators
- ★ Fourier transformation
- ★ Pyramids and wavelets
- ★ Geometric transformations

Image Processing

- ★ An image is a **spatial representation** of a two dimensional or three- dimensional scene.
- ★ A digital image is composed of a finite number of elements, each of which has a particular location and value.
- ★ A digital image are classified into **three** types,
 - Black/White or Binary image
 - Gray-scale image
 - Color/RGB image
- ★ Image processing takes an image as input and return an image as output.

Image Processing

Some Image Processing Operations



(a) original image



(b) increased contrast



(c) change in hue



(d) “posterized” (quantized colors)



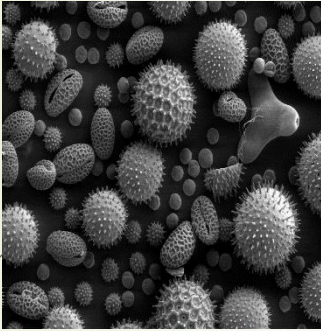
(e) blurred



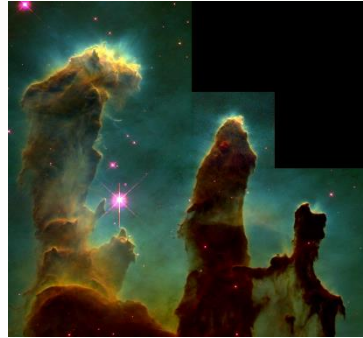
(f) rotated

Image Processing

Application of Image Processing



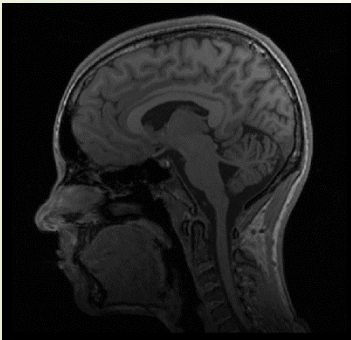
Biology



Astronomy



Security, Biometric



Medical Imaging



Satellite Imagery



Personal Photos

Image Processing

Basic Digital Image Processing Techniques

- ★ Image Enhancement
- ★ Image Restoration
- ★ Image Compression
- ★ Image Segmentation

Image Processing

Image Enhancement

- ★ Image enhancement is the process of **adjusting digital images** so that the results are more suitable for display or further image analysis.
- ★ Image enhancement methods operate in,
 - **Spatial domain**
 - manipulating the pixel data
 - **Frequency domain**
 - modifying the spectral component
- ★ Spatial domain methods operate **directly on the pixels**.

Image Processing

Spatial Domain Process

- ★ Two principal categories of spatial domain processing are
 - Point processing or Intensity Transformation
 - Spatial Filtering
- ★ *Intensity transformation* operate on single pixels of an image, principally for the purpose of contrast manipulation and image threshold.
- ★ *Spatial filtering* deals with performing operations, such as image sharpening, by working in a neighborhood of every pixel in an image.

Image Processing

Spatial Domain Process

★ The spatial domain process can be denoted by the expression,

$$g(x, y) = T[f(x, y)]$$

Here,

- x and y are spatial (plane) coordinates
- $f(x, y)$ is the intensity/gray level the image at that point
- $f(x, y)$ is the input image
- $g(x, y)$ is the output image
- T is an operator

Point Operators

- ★ **Point operators** are image processing operations that apply the same mathematical operation to each pixel in an image.
- ★ It generates an output pixel based on a single input pixel.
- ★ It change the distribution and grayscale range of an image.
- ★ It includes thresholding, color-space conversion, histogram equalization, and image arithmetic.
- ★ It don't change the image's structure or size. Each new pixel value is only based on the original value at that point.

Point Operators

Point Processing Function

- ★ The point processing is a simple method of **image enhancement**.
- ★ The point processing technique, which results depend **only on the intensity at a point**.
- ★ This point processing function of the form

$$s = T (r)$$

Here,

- s refers to the processed image
- r refers to the original image
- T is a transformation that maps a pixel value r into a pixel value s

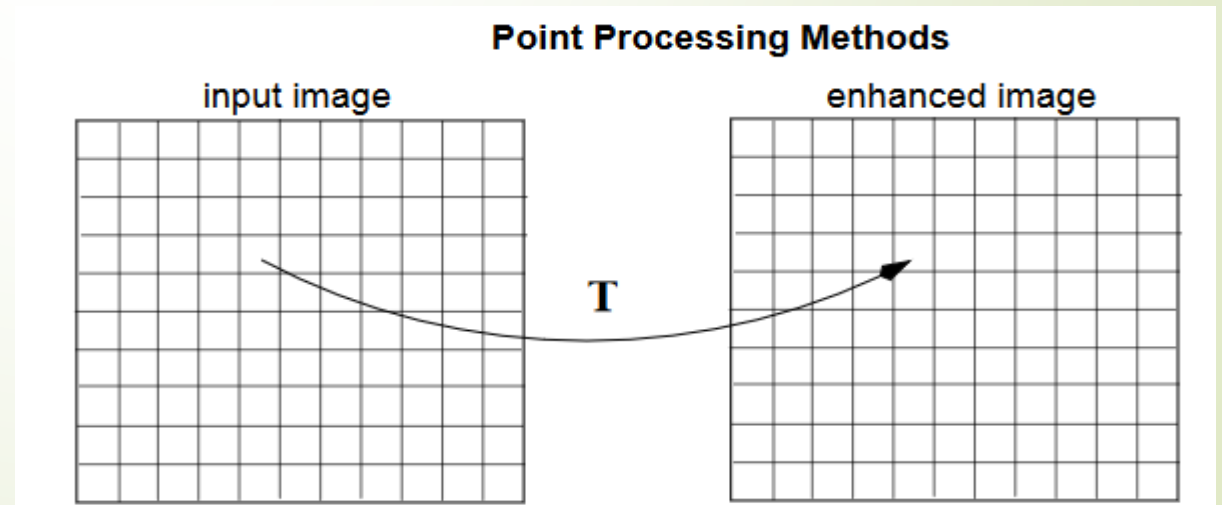
Point Operators

Point Processing Function

- ★ The point processing function is also called as **intensity transformation** or **gray-level mapping function**.
- ★ The point processing functions can be further categorized into two types.
 - Linear Functions
 - Non – Linear Functions
- ★ Point operations are defined as functions that are performed on each pixel of an image, **independent of all other pixels** in that image.

Example of operations:

brightness,
contrast adjustments,
color correction and
transformations



Point Operators

Pixel Transforms

- ★ Image processing operator is a function that takes one or more input images and produces an output image

$$g(x)=h(f(x)) \quad \text{or} \quad g(x)=h(f_0(x),\dots,f_n(x))$$

- x is in the D-dimensional domain of the functions
- The function f and g operate over some range
- Domain consists of a finite number of pixel locations, $x=(i, j)$ $g(i,j)=h(f(i,j))$

- ★ Two commonly used point processes are : **Multiplication** and **Addition**

$$g(x)= a f(x)+b$$

- The parameters $a>0$ and b are called the gain and bias parameters
- To control contrast and brightness
- The bias and gain parameters can be spatially varying

$$g(x) =a(x) f(x) +b(x)$$

Point Operators

Pixel Transforms

- ★ Multiplicative gain is a linear operation

$$h(f_0 + f_1) = h(f_0) + h(f_1)$$

- ★ Dyadic (two-input) operator is the linear blend operator,

$$g(\mathbf{x}) = (1 - \alpha)f_0(\mathbf{x}) + \alpha f_1(\mathbf{x})$$

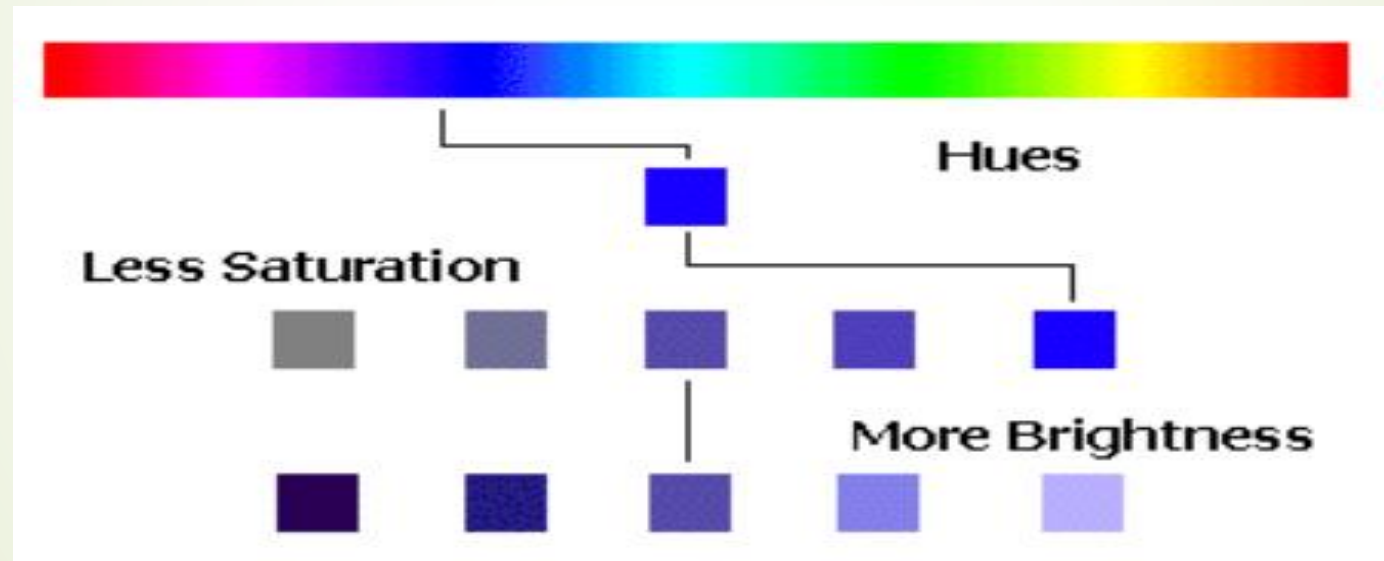
- ★ Non-linear transform: gamma correction

$$g(\mathbf{x}) = [f_0(\mathbf{x})]^{1/\gamma}$$

Point Operators

Color Transforms

- ★ Adding the same value to each color channel not only increases the apparent intensity of each pixel, it can also affect the pixel's hue and saturation.



Point Operators

Composition and Matting

★ Matting

- The process of extracting the object from the original image.

★ Composition

- The process of inserting it into another image.

★ Alpha-Matted color image

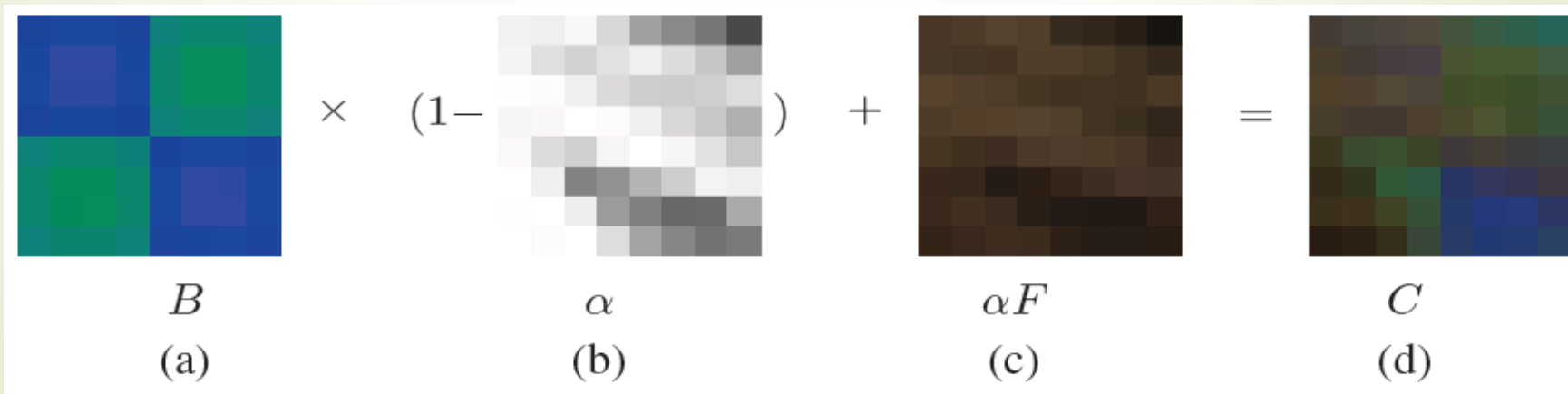
- The **intermediate representation** used for the foreground object between these two stages
- In addition to the three color RGB channels, an alpha-matted image contains a fourth **alpha channel a or A** that describes the relative amount of opacity

Point Operators

Composition and Matting



(a) source image; (b) extracted foreground object F ; (c) alpha matte a (d) new composite C



The diagram illustrates the composite equation $C = (1 - a) B + a F$ using four small square images labeled (a) through (d). Image (a) is a 2x2 grid of blue and green squares, labeled B below it. Image (b) is a grayscale alpha matte, labeled α below it. Image (c) is the result of multiplying image (a) by $(1 - \alpha)$, labeled αF below it. Image (d) is the final composite image C , which is the sum of image (c) and image (b), labeled C below it. The equation is represented by the sequence of images and operators: $B \times (1 - \alpha) + \alpha F = C$.

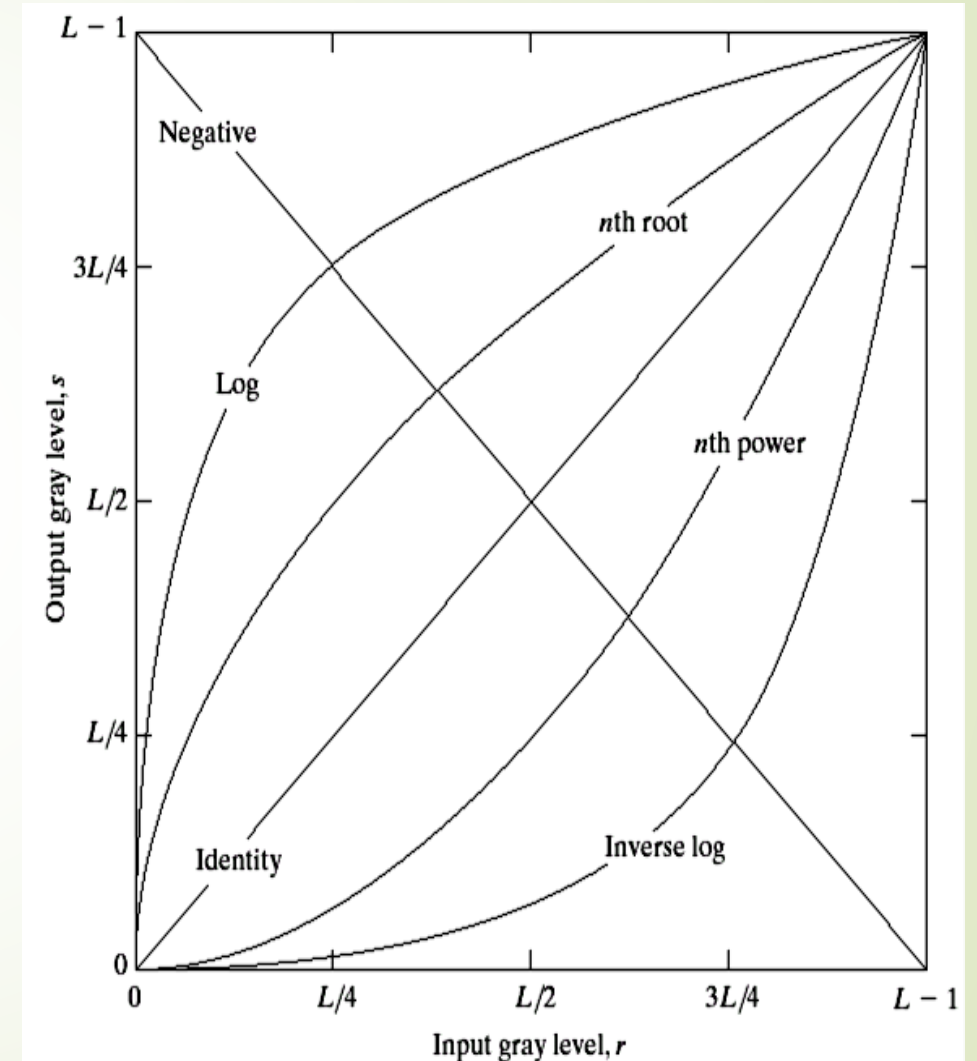
Composite equation $C = (1 - a) B + a F$

Point Operators

- ★ Point operators are image processing operations that apply the same mathematical operation to each pixel in an image.
- ★ It generates an output pixel based on a single input pixel.
- ★ It change the distribution and grayscale range of an image.
- ★ It includes thresholding, color-space conversion, histogram equalization, and image arithmetic.
- ★ It don't change the image's structure or size. Each new pixel value is only based on the original value at that point.

Basic Gray Level Transformations

- ★ Different kinds of gray level transformations
- ★ Three of the most common are:
 - Linear transformation
Negative/Identity
 - Logarithmic transformation
Log/Inverse log
 - Power law transformation
 n^{th} power/ n^{th} root



Basic Gray Level Transformations

Negative transformations

- ★ Negative images are useful for *enhancing white or gray detail embedded* in dark regions of an image.
- ★ Negative transform exchanges dark values for light values and vice versa.
- ★ The Negative Transformations can be defined by

$$s = (L - 1 - r)$$

- ★ Negative of an image intensity levels in the range $[0, L-1]$,

$L-1$ = Maximum pixels value

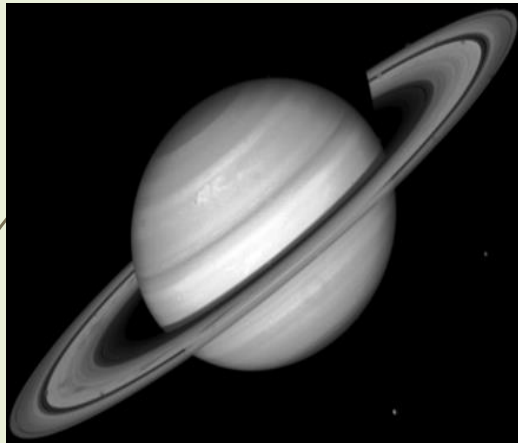
s = output pixel value

r = input pixel value

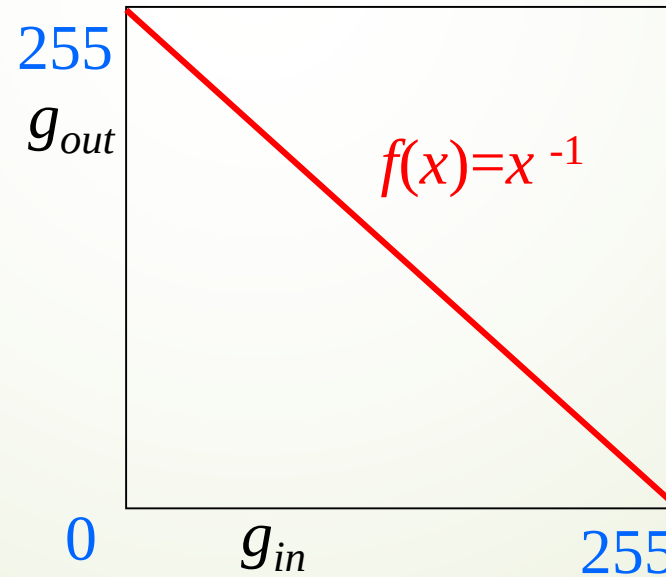
Basic Gray Level Transformations

★ Negative transformations

Input image



Mapping function (Reverse)



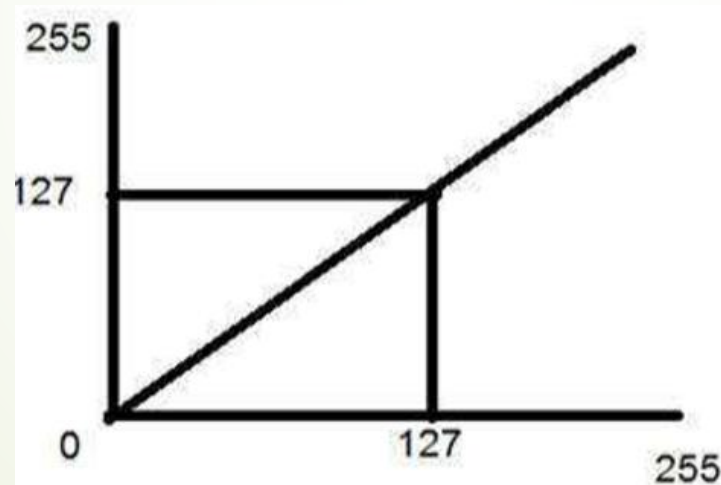
Output image



Basic Gray Level Transformations

Identity Transformation

- ★ Each value of the input image is directly mapped to each other value of output image.
- ★ That results in the same input image and output image.



Graph representation

Basic Gray Level Transformations

Log Transformation

- ★ This transform is used to *expand values of dark pixels* and *compress values of bright pixels*.
- ★ The log transformations can be defined by this formula

$$s = c \log(r+1)$$

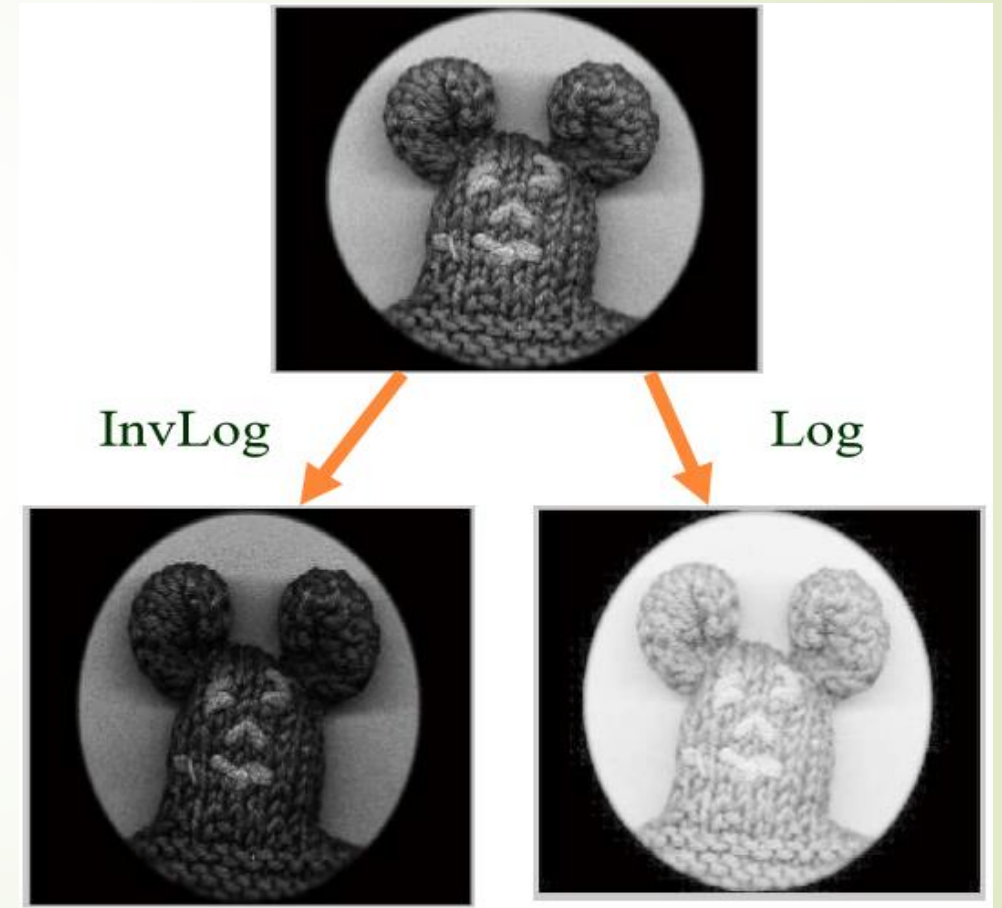
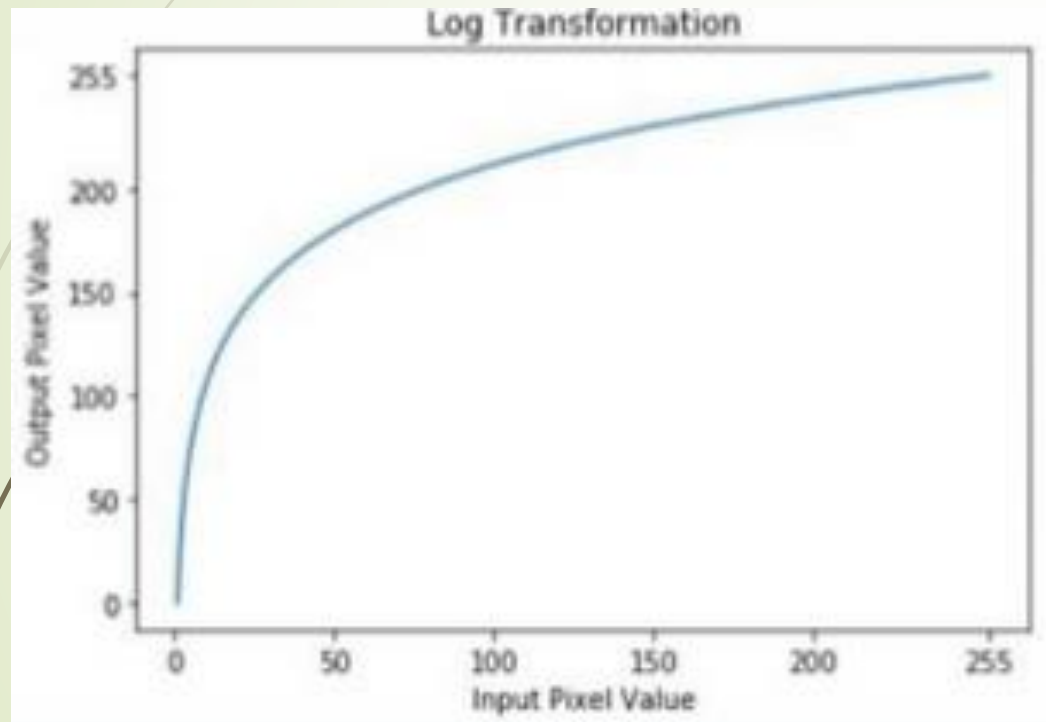
s and r = pixel values of the output and the input image

c = constant.

- ★ The log transformation maps a narrow range of low input gray level values into a wider range of output values.
- ★ The inverse log transformation performs the opposite transformation.

Basic Gray Level Transformations

★ Log Transformation



Basic Gray Level Transformations

Power Law Transformation

- ★ This transformation is used for *enhancing images for different types of display devices*.
- ★ These transformations can be given by,

$$s = c r^\gamma$$

Here, s = output pixel value

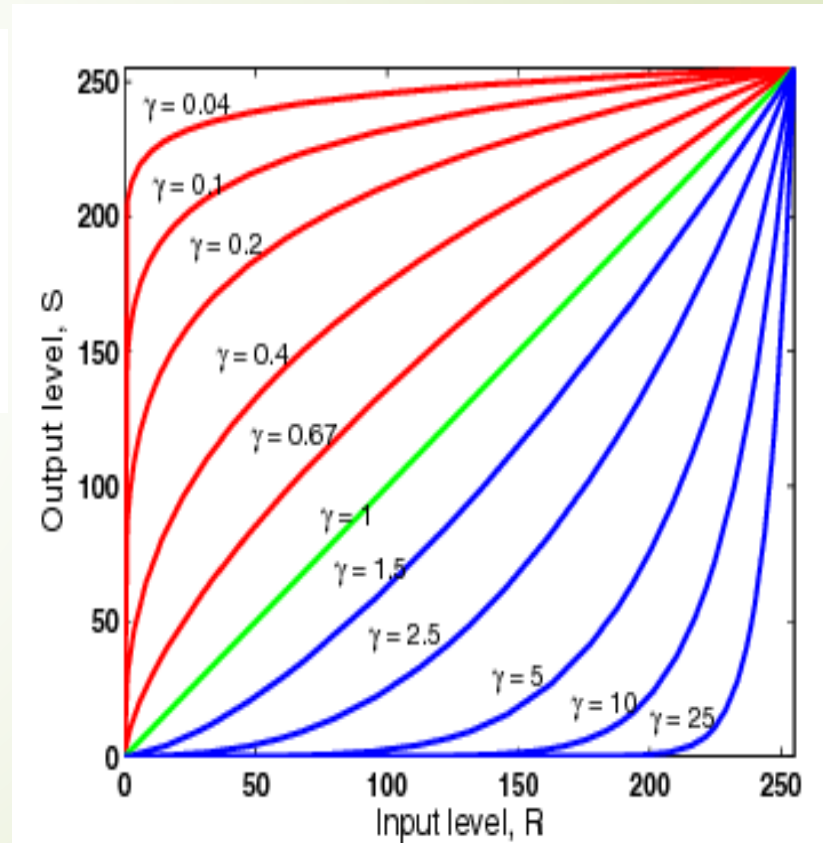
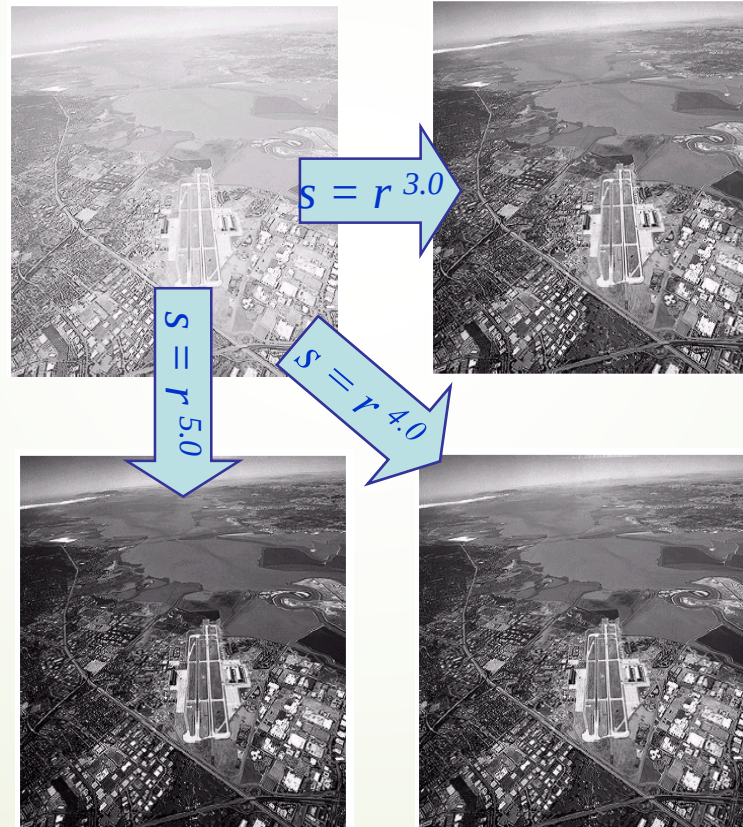
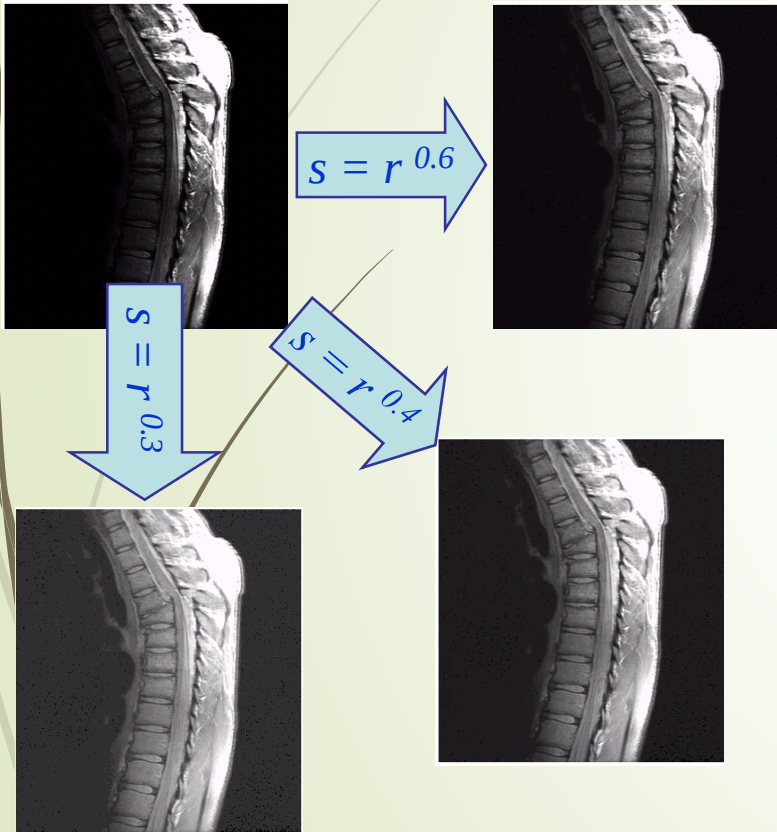
r = input pixel value,

c and γ = real numbers.

- ★ Variation in the value of γ varies the enhancement of the images. This technique is commonly called as *Gamma correction*.

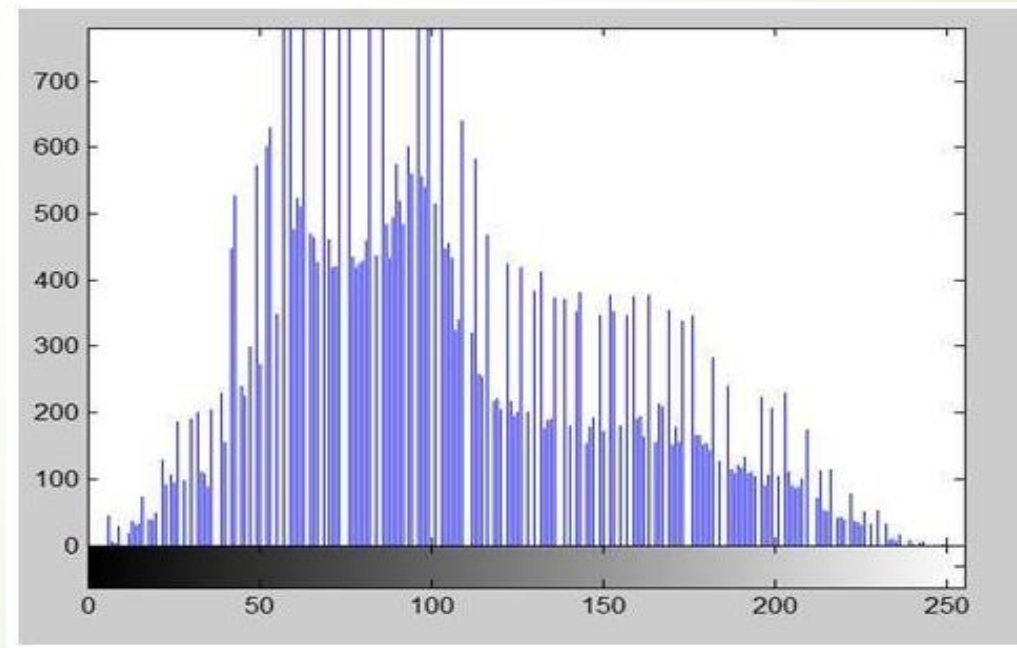
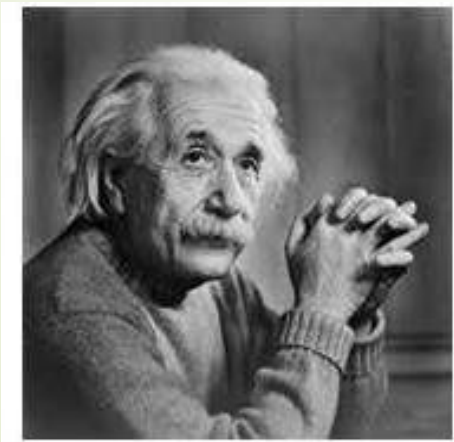
Basic Gray Level Transformations

★ Power Law Transformation



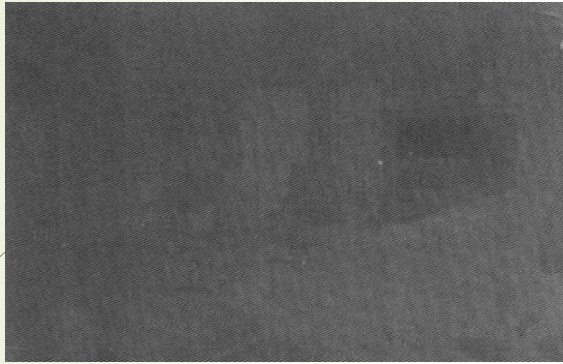
Histogram Equalization

- ★ An image histogram shows frequency of pixels intensity values.
- ★ In an image histogram,
 - the *x axis* shows the *gray level* intensities and
 - the *y axis* shows the *frequency* of these intensities (number of pixels).

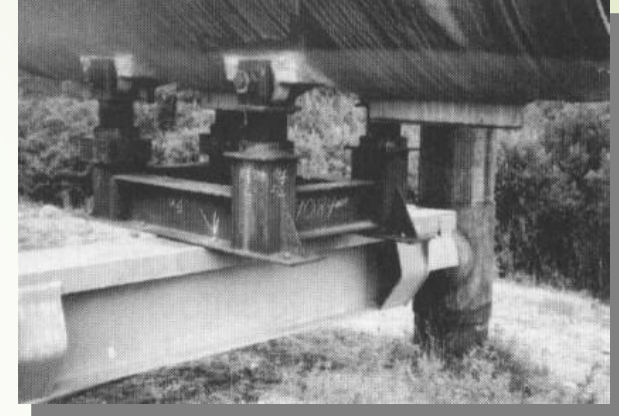
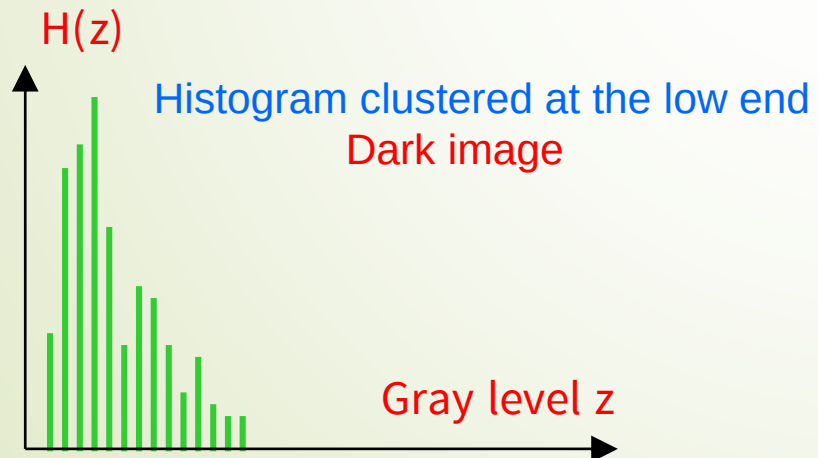


Histogram Equalization

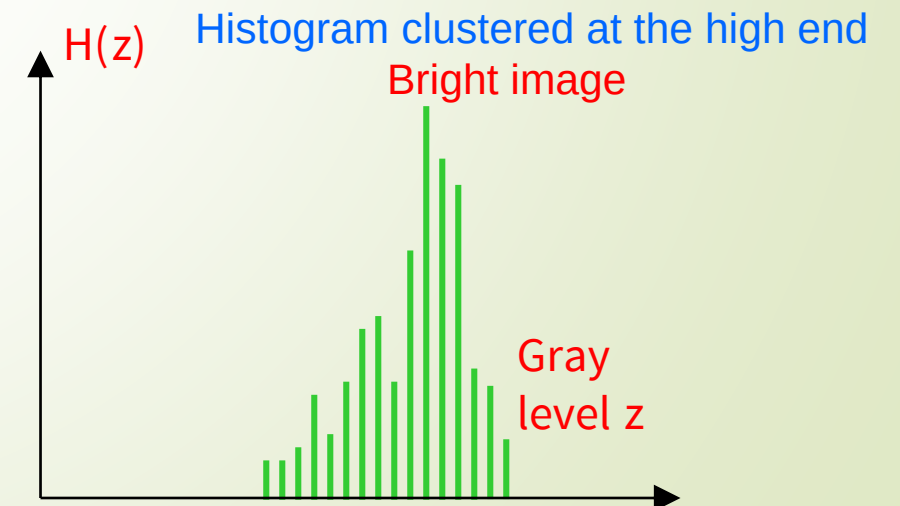
★ Histograms of Four Basic types of images



Dark image

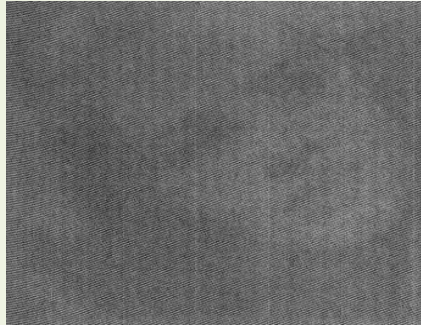


Bright image

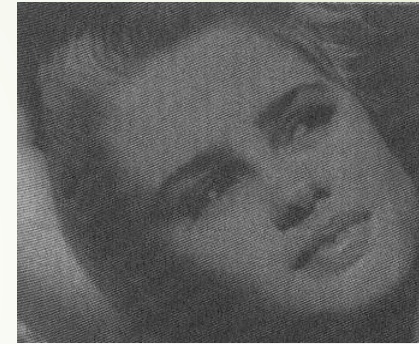


Histogram Equalization

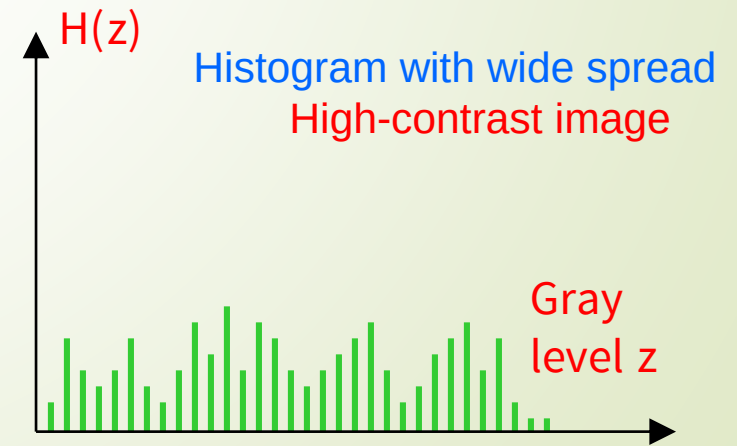
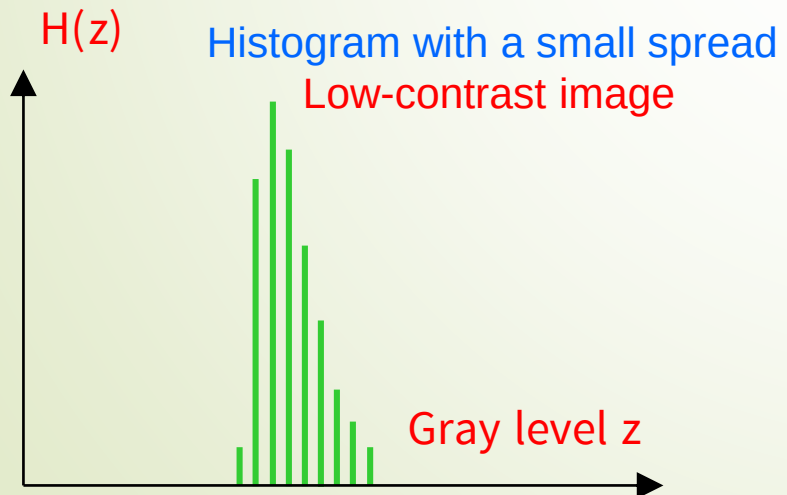
★ Histograms of Four Basic types of images



Low-contrast image



High-contrast image



Histogram Equalization

- ☆ The histogram of the *bad* image is very *narrow* (Fig. a),
- ☆ The histogram of the *good* image is *more spread* (Fig. b).
- ☆ To change a bad image to the good image, we need modify the histogram.

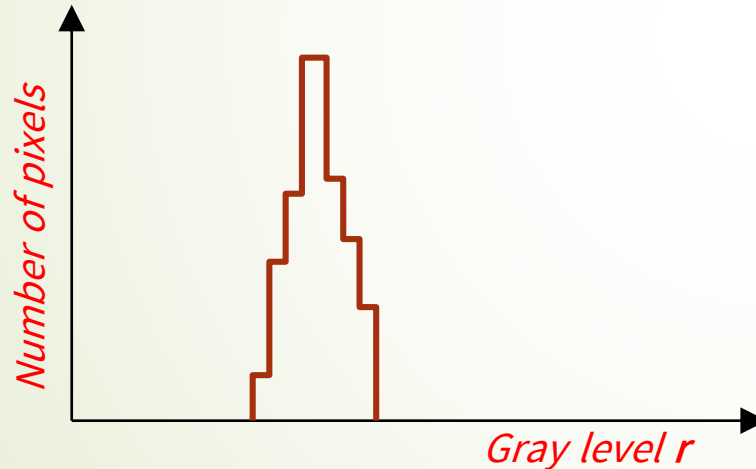


Figure (a) . bad image

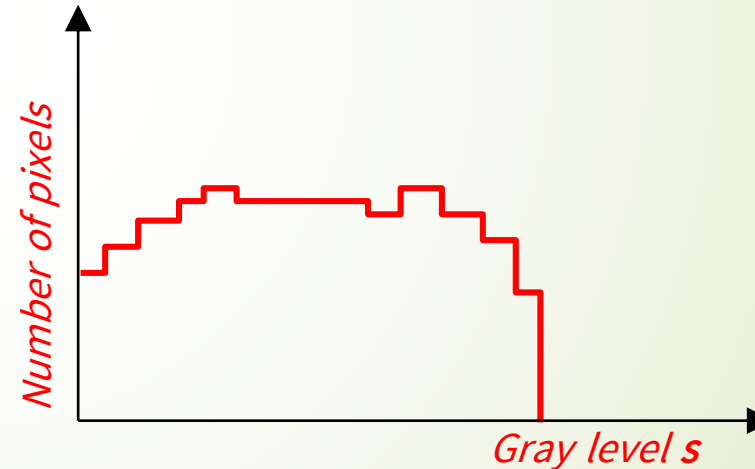
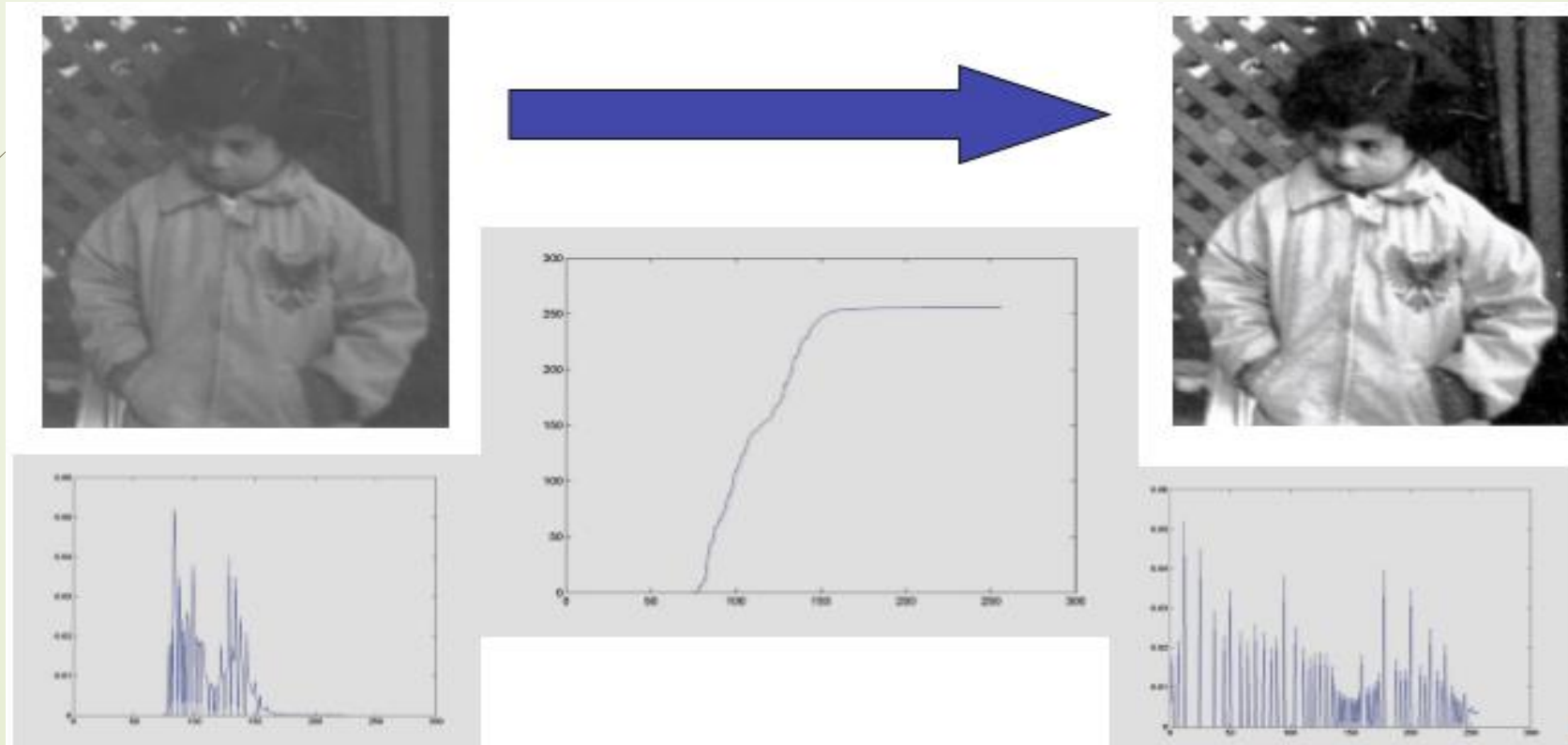


Figure (b). good image

Histogram Equalization

- ★ Histogram equalization is an important method in histogram modification.
- ★ Histogram equalization is often tried to *enhance an image*.



Histogram Equalization

Consider a discrete gray scale image $\{x\}$

★ Histogram is a discrete function $P_x(i) = \frac{n_i}{n}$

n_i = the number of occurrences of gray level i

n = total number of pixels

$P_x(i)$ = probability of an occurrence of a pixel of level i

★ Also define the **cumulative distribution function (CDF)** corresponding to P_x is

$$cdf_x(i) = \sum_{j=0}^i P_x(x = j)$$

★ To produce a new image $\{y\}$ is $CDF * (Levels-1)$

Histogram Equalization

★ Example

Perform the histogram equalization on the following 4×4 image. The gray level distribution of image is given below: (intensity level from 0 to 20)

3	2	4	5
7	7	0	2
3	1	2	3
5	4	6	7

Histogram Equalization

3	2	4	5
7	7	0	2
3	1	2	3
5	4	6	7



10	6	12	15
20	20	1	6
10	2	6	10
15	12	16	20

Gray level (i)	0	1	2	3	4	5	6	7
No. of pixels (n_i)	1	1	3	3	2	2	1	3
Probability $P_x(i)$	0.0625	0.0625	0.1875	0.1875	0.125	0.125	0.0625	0.1875
Cumulative Probability	0.0625	0.125	0.3125	0.5	0.625	0.75	0.8125	1
CP * 20	1.25	2.5	6.25	10	12.5	15	16.25	20
Floor round	1	2	6	10	12	15	16	20

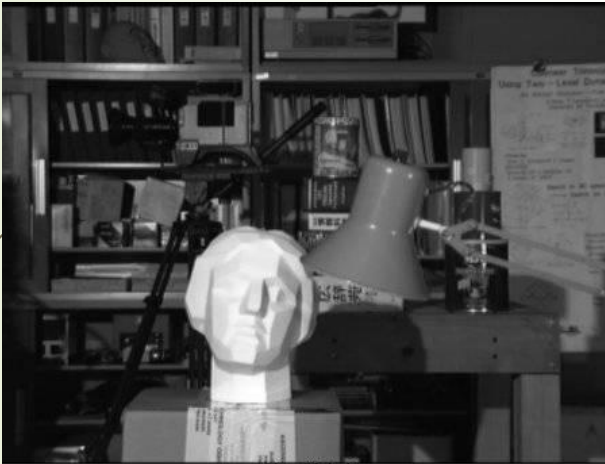
The intensity range of the pixels has been increased

Histogram Equalization

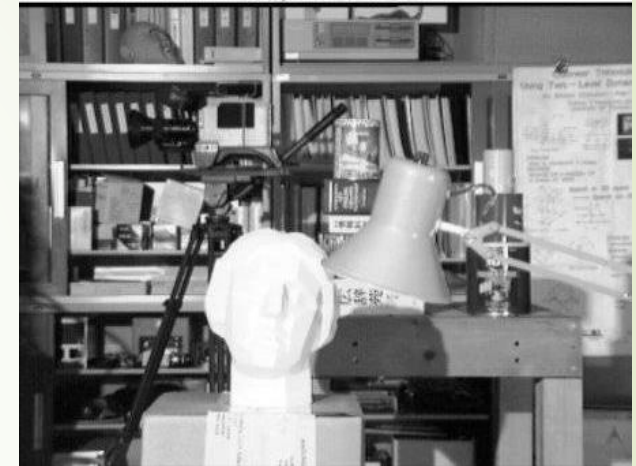
- ★ Adaptive histogram equalization (AHE)
- ★ Some images might be preferable to apply different kinds of equalization in different regions.
- ★ The adaptive method computes several histograms, each corresponding to a distinct section of the image.
- ★ Instead of computing a single curve, the image was subdivided into $M \times M$ pixel blocks and perform separate histogram equalization in each sub-block.
- ★ It is suitable for *improving the local contrast*.

Histogram Equalization

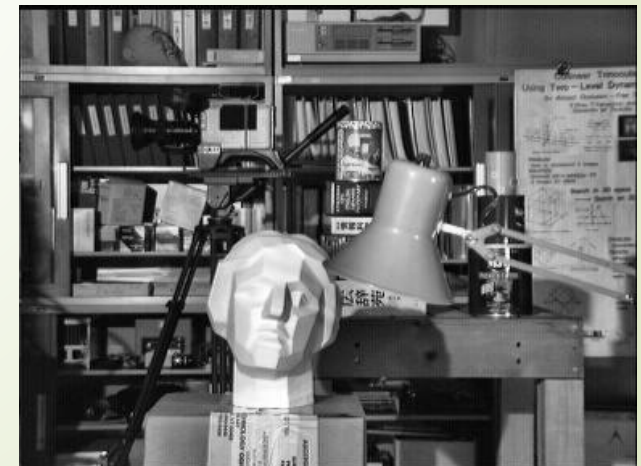
Original image



After histogram equalization



After AHE



- ★ Histogram would confine to a small region (unless there is noise).
- ★ If noise is there, it will be amplified.
- ★ To avoid this, **Contrast Limited AHE** is applied.

Image Filtering

- ★ For each pixel, compute function of local neighborhood and output a new value.
- ★ Image filtering is useful for many applications, including smoothing, sharpening, removing noise, and edge detection, etc.
- ★ Image filters can be classified as *linear* or *nonlinear*.
- ★ Linear filters are also known as *convolution filters* as they can be represented using a matrix multiplication.
- ★ Thresholding and image equalization are examples of nonlinear operations, as is the median filter.

Linear Filtering

- ★ It is filtering in which the value of an output pixel is a linear combination of the values of the pixels in the input pixel's neighborhood.
- ★ The linear filter works best with salt and pepper noise, and Gaussian noise.
- ★ Linear filter is the most common type, in which an output pixel's value (i.e. $g(i, j)$) is determined as a weighted sum of input pixel values (i.e. $f(i+k, j+l)$).

$$g(i, j) = \sum_{k, l} f(i + k, j + l) h(k, l)$$

- ★ $h(k, l)$ is called the kernel and different choices of h lead to filters that smooth, sharpen, and detect edges, to name a few applications.

Linear Filtering

Convolution

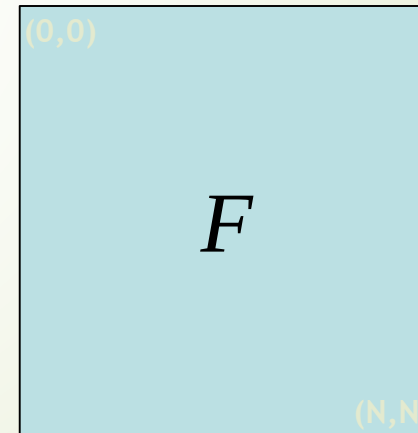
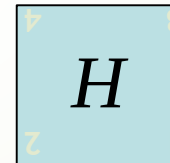
- ★ Flip the filter in both dimensions (bottom to top, right to left)
- ★ Then apply cross-correlation

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

$$G = H \star F$$



*Notation for
convolution
operator*



Linear Filtering

Convolution

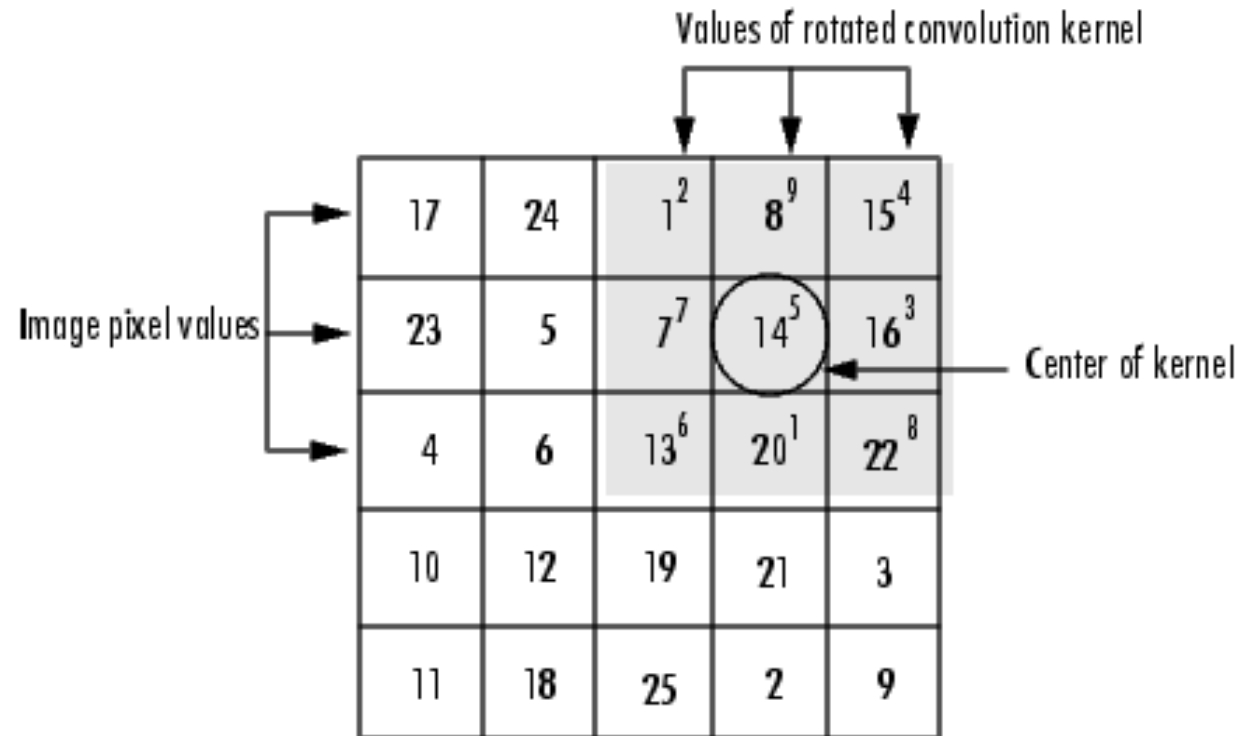
- ★ Linear filtering of an image is accomplished through an operation called *convolution*.
- ★ In convolution, the value of an output pixel is computed as a weighted sum of neighboring pixels.
- ★ The matrix of weights is called the *convolution kernel*, also known as the *filter*.
- ★ To compute the output pixel – $A(2,4)$
 1. Rotate the convolution kernel 180 degrees about its center element.
 2. Slide the center element of the convolution kernel so that lies on top of the (2,4) element of A.
 3. Multiply each weight in the rotated convolution kernel by the pixel of A underneath.
 4. Sum up the individual products from step 3.

Linear Filtering

Convolution

$A = \begin{bmatrix} 17 & 24 & 1 & 8 & 15 \\ 23 & 5 & 7 & 14 & 16 \\ 4 & 6 & 13 & 20 & 22 \\ 10 & 12 & 19 & 21 & 3 \\ 11 & 18 & 25 & 2 & 9 \end{bmatrix}$

$h = \begin{bmatrix} 8 & 1 & 6 \\ 3 & 5 & 7 \\ 4 & 9 & 2 \end{bmatrix}$



$$1 \cdot 2 + 8 \cdot 9 + 15 \cdot 4 + 7 \cdot 7 + 14 \cdot 5 + 16 \cdot 3 + 13 \cdot 6 + 20 \cdot 1 + 22 \cdot 8 = 575$$

Linear Filtering

Convolution

Convolution kernel, ω

1	-1	-1
1	2	-1
1	1	1

Rotate 180°

1	1	1
-1	2	1
-1	-1	1

Input Image, f

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

Linear Filtering

Convolution

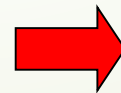
1	1	1
-1	2	1
-1	-1	1

Convolution

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

1	1	1		
-1	4	2	2	3
-1	-2	1	3	3
	2	2	1	2
	1	3	2	2

Input
Image, f



5			

Output
Image, g

Linear Filtering

Convolution

1	1	1
-1	2	1
-1	-1	1

1	1	1	
-2	4	2	3
-2	-1	3	3
2	2	1	2
1	3	2	2

Input Image, f

Convolution

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

5	4		

Output Image, g

Linear Filtering

Convolution

1	1	1
-1	2	1
-1	-1	1

		1	1	1
2	-2	4	3	
2	-1	-3	3	
2	2	1	2	
1	3	2	2	

Input Image, f

Convolution

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

5	4	4	

**Output
Image, g**

Linear Filtering

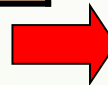
Convolution

1	1	1
-1	2	1
-1	-1	1

Convolution

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

		1	1	1
2	2	-2	6	1
2	1	-3	-3	1
2	2	1	2	
1	3	2	2	



5	4	4	-2

Output
Image, g

Input Image, f

Linear Filtering

Convolution

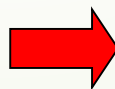
1	1	1
-1	2	1
-1	-1	1

Convolution

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

1	2	2	2	3
-1	4	1	3	3
-1	-2	2	1	2
	1	3	2	2

Input Image, f



5	4	4	-2
9			

Output
Image, g

Linear Filtering

Convolution

Convolution

5	4	4	-2
9	6	14	5
11	7	6	5
9	12	8	5

Final output Image, g

Linear Filtering

Correlation

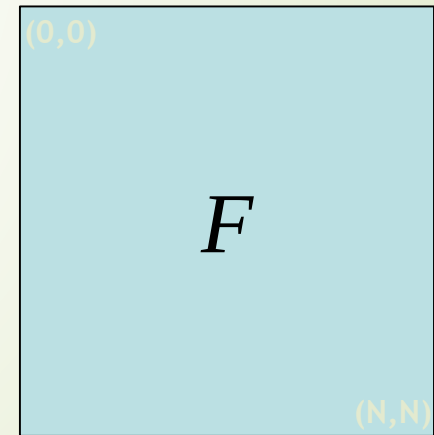
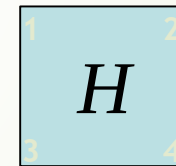
★ The filter **kernel** or **mask** is the prescription for the weights in the linear combination.

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

$$G = H \otimes F$$



***Notation for
correlation
operator***



Linear Filtering

Correlation

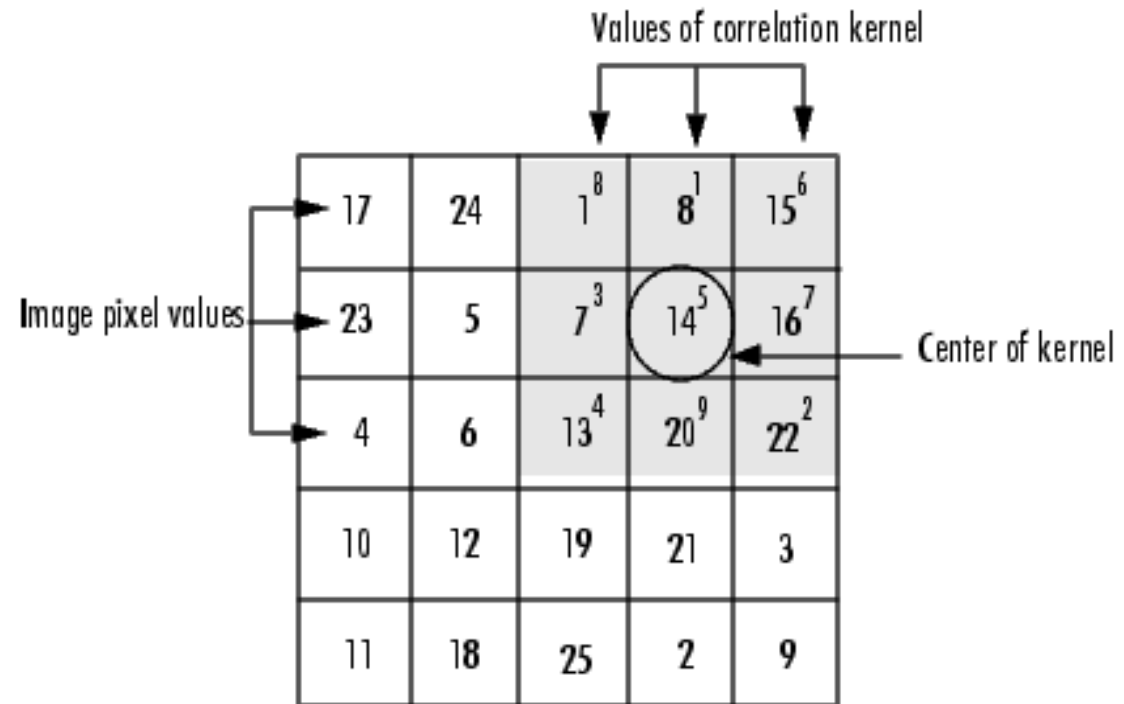
- ★ In correlation, the value of an output pixel is also computed as a **weighted sum** of **neighboring pixels**.
- ★ The difference is that the matrix of weights, in this case called the **correlation kernel**, is **not rotated** during the computation.
- ★ To compute the output pixel – $A(2,4)$
 1. Slide the center element of the correlation kernel so that lies on top of the (2,4) element of A.
 2. Multiply each weight in the correlation kernel by the pixel of A underneath.
 3. Sum up the individual products from step 2.

Linear Filtering

Correlation

$A = \begin{bmatrix} 17 & 24 & 1 & 8 & 15 \\ 23 & 5 & 7 & 14 & 16 \\ 4 & 6 & 13 & 20 & 22 \\ 10 & 12 & 19 & 21 & 3 \\ 11 & 18 & 25 & 2 & 9 \end{bmatrix}$

$h = \begin{bmatrix} 8 & 1 & 6 \\ 3 & 5 & 7 \\ 4 & 9 & 2 \end{bmatrix}$



$$1 \cdot 8 + 8 \cdot 1 + 15 \cdot 6 + 7 \cdot 3 + 14 \cdot 5 + 16 \cdot 7 + 13 \cdot 4 + 20 \cdot 9 + 22 \cdot 2 = 585$$

Linear Filtering

Correlation

correlation kernel, ω

1	-1	-1
1	2	-1
1	1	1

Don't rotate use it directly

Input Image f

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

Linear Filtering

Correlation

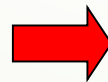
1	-1	-1
1	2	-1
1	1	1

1	-1	-1
1	4	-2
1	2	1

2	2	1	2
1	3	2	2

Input Image, f

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2



5			

**output
Image, g**

Linear Filtering

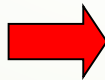
Correlation

1	-1	-1
1	2	-1
1	1	1

1	-1	-1	
2	4	-2	3
2	1	3	3
2	2	1	2
1	3	2	2

Input Image, f

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2



5	10		

**output
Image, g**

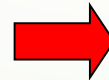
Linear Filtering

Correlation

1	-1	-1
1	2	-1
1	1	1

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

		1	-1	-1	
2	2	4	-3		
2	1	3	3		
2	2	1	2		
1	3	2	2		



5	10	10	

Input Image, f

**output
Image, g**

Linear Filtering

Correlation

1	-1	-1
1	2	-1
1	1	1

		1	-1	-1
2	2	2	6	-1
2	1	3	3	1
2	2	1	2	
1	3	2	2	

Input Image, f

Correlation

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

5	10	10	15

output
Image, g

Linear Filtering

Correlation

5	10	10	15
3	4	6	11
7	11	4	9
-5	4	4	5

Linear Filtering

Key Properties of Linear Filters

- ★ **Linearity:** $\text{filter}(f_1 + f_2) = \text{filter}(f_1) + \text{filter}(f_2)$
- ★ **Shift invariance:** same behavior regardless of pixel location:
$$\text{filter}(\text{shift}(f)) = \text{shift}(\text{filter}(f))$$
- ★ Any linear shift-invariant operator can be represented as a convolution
- ★ **Commutative:** $a * b = b * a$
 - Conceptually no difference between filter and signal
- ★ **Associative:** $a * (b * c) = (a * b) * c$
 - Often apply several filters one after another: $((a * b_1) * b_2) * b_3$
 - This is equivalent to applying one filter: $a * (b_1 * b_2 * b_3)$
- ★ **Distributes over addition:** $a * (b + c) = (a * b) + (a * c)$
- ★ **Scalars factor out:** $ka * b = a * kb = k(a * b)$
- ★ **Identity:** unit impulse $e = [0, 0, 1, 0, 0]$, $a * e = a$

Linear Filtering

Shift Invariant Linear System

★ Shift invariant:

- Operator behaves the same everywhere, *i.e.* the value of the output depends on the pattern in the image neighborhood, not the position of the neighborhood.

★ Linear :

- Superposition: $h * (f_1 + f_2) = (h * f_1) + (h * f_2)$
- Scaling: $h * (kf) = k(h * f)$

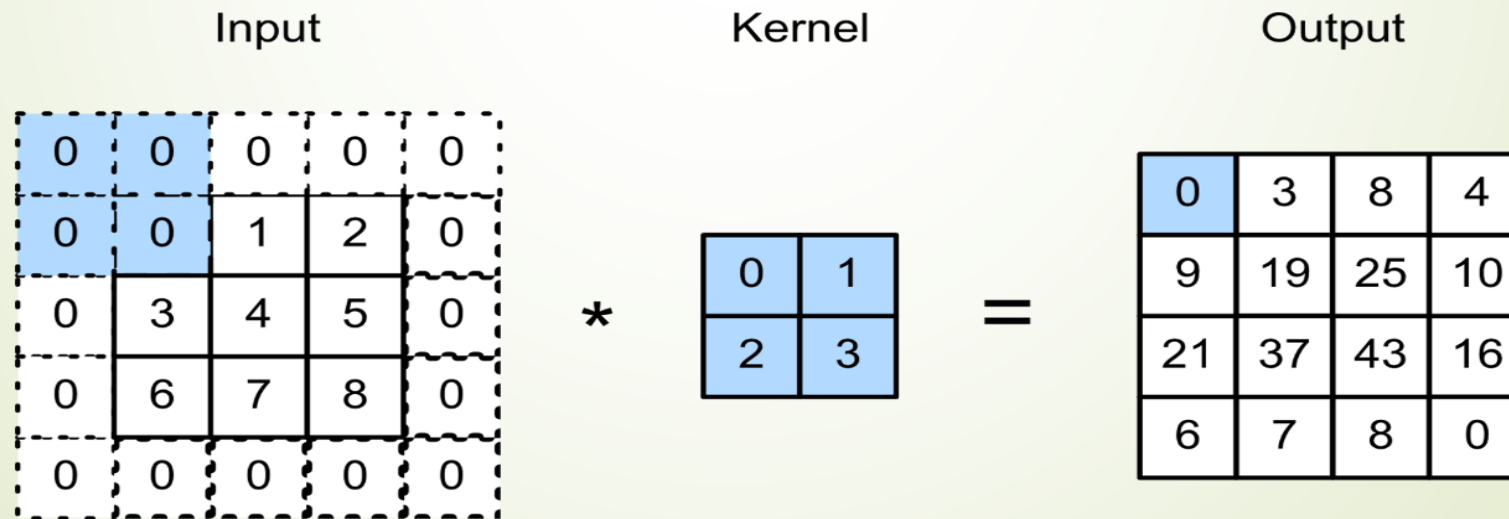
Linear Filtering

Problem with Simple Convolution Layers

- ★ For a gray scale $(n \times n)$ image and $(f \times f)$ filter/kernel, the dimensions of the image resulting from a convolution operation is

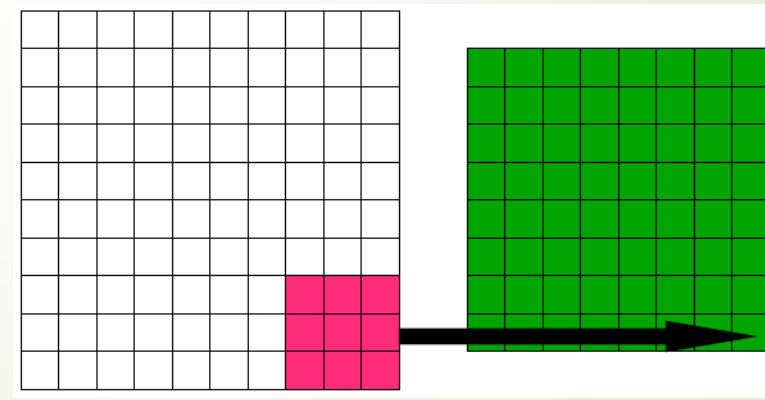
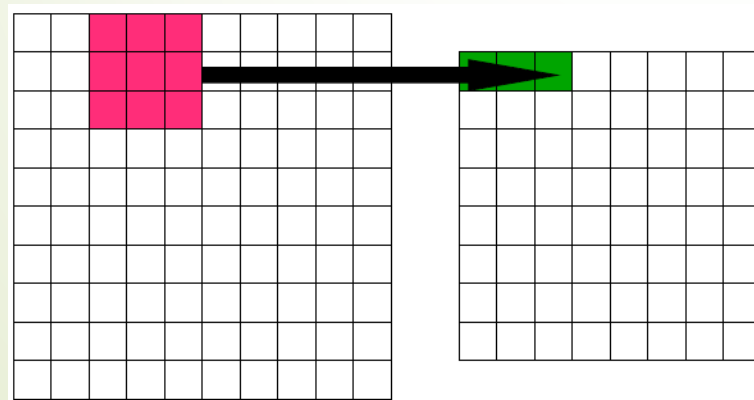
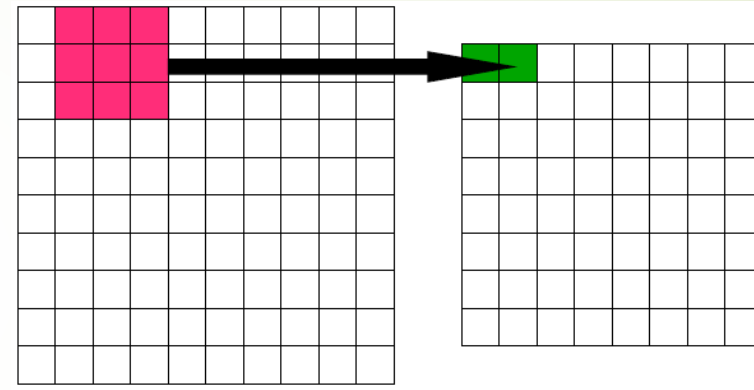
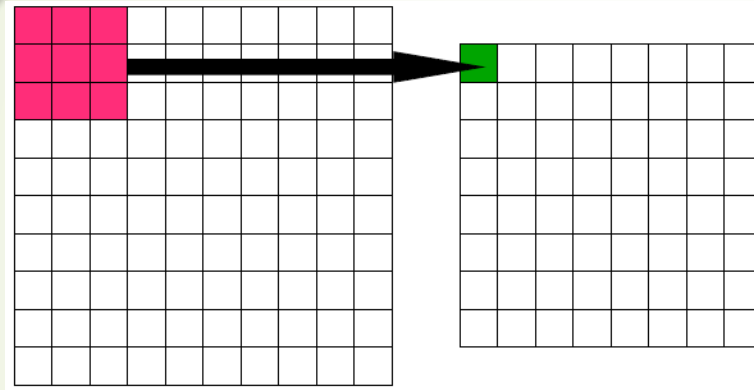
$$(n - f + 1) \times (n - f + 1)$$

- ★ For example, for an (5×5) image and (2×2) filter, the output resulting after convolution operation would be of size (4×4) .



Linear Filtering

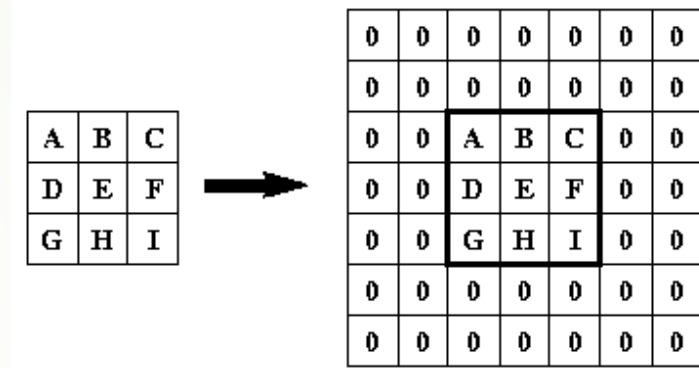
Problem with Simple Convolution Layers



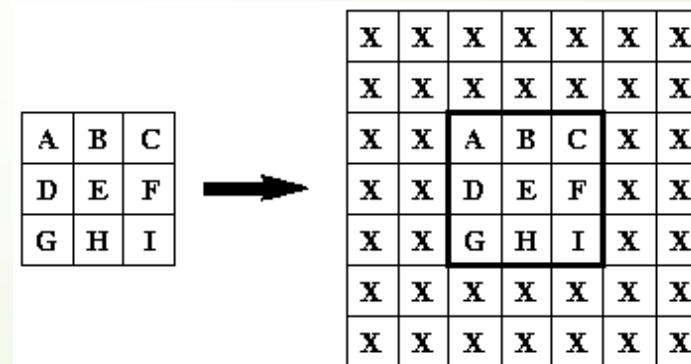
Linear Filtering

Alternative Padding

1. **zero**: set all pixels outside the source image to 0.



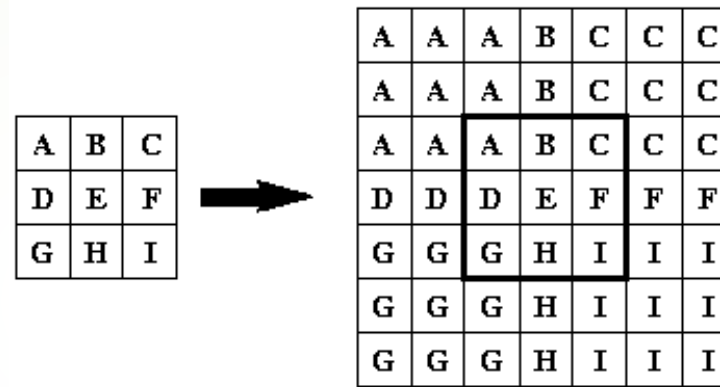
2. **constant** (border color): set all pixels outside the source image to a specified border value;



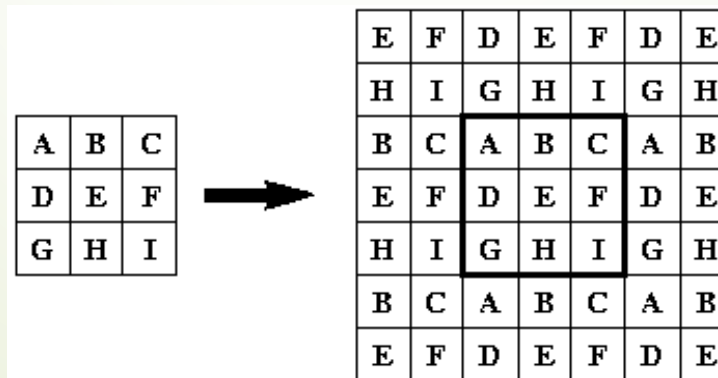
Linear Filtering

Alternative Padding

3. **clamp** (replicate or clamp to edge): repeat edge pixels indefinitely;



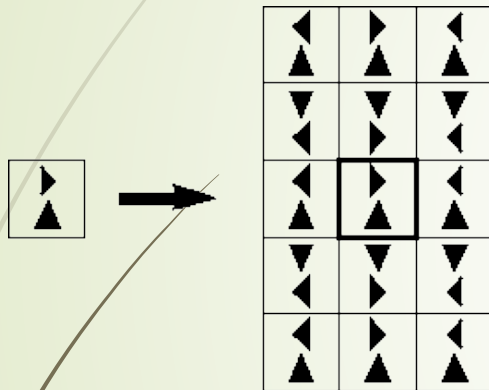
4. **wrap** (repeat or tile): loop “around” the image in a “toroidal” configuration;



Linear Filtering

Alternative Padding

5. **mirror**: reflect pixels across the image edge;



3	5	1
3	6	1
4	7	9

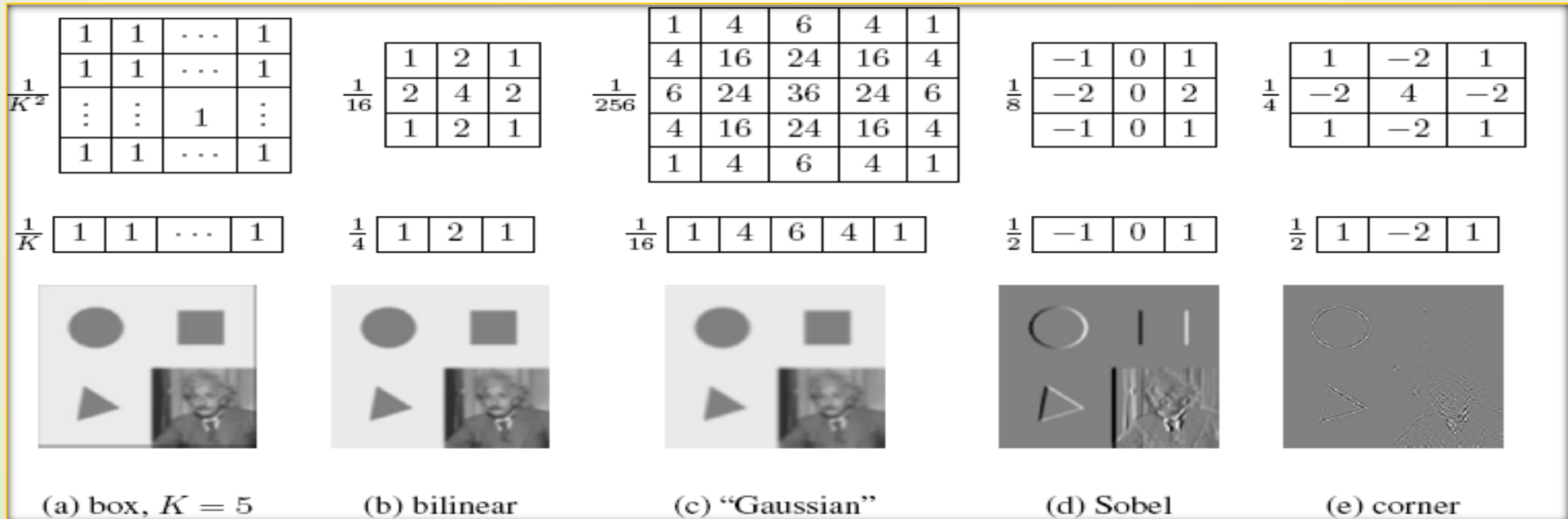
1	6	3	6	1	6	3
1	5	3	5	1	5	3
1	6	3	6	1	6	3
9	7	4	7	9	7	4
1	6	3	6	1	6	3

6. **extend**: extend the signal by subtracting the mirrored version of the signal from the edge pixel value.

Linear Filtering

Separable Filtering

- ☆ A separable filter in image processing can be written as **product of two more simple filters**.
- ☆ Typically a 2-dimensional convolution operation is separated into two 1-dimensional filters.
- ☆ This reduces the cost of computing the operator.
- ☆ Process of performing a **convolution requires K^2 (multiply-add)** operations per pixel.
- ☆ This operation can be significantly **sped up** by first performing a one-dimensional horizontal convolution followed by a one-dimensional vertical convolution (**which requires a total of $2K$ operations per pixel**).



Linear Filtering

Example of Linear Filtering

(a) Box

(b) bilinear

(c) Gaussian

(d) Sobel

(e) Corner

} low-pass filters (or) blurring/smoothing

} band-pass and oriented filters.

Linear Filtering

Box Filter

- ★ Averaging is done by convolving image with a normalized box filter.
- ★ It simply takes the average of all the pixels under kernel area and replace the central element.
- ★ We should specify the width and height of kernel.
- ★ A Box filter could be of dimensions 3x3, 5x5, 9x9 etc.
- ★ A 3x3 normalized box filter would look like below:

$$K = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

[illegible]

$$g[\cdot, \cdot]^{-\frac{1}{9}}$$

1	1	1
1	1	1
1	1	1

[illegible]

Linear Filtering

Box Filter

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0	10							

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

Box Filter

[illegible]

Box Filter

[illegible]

Box Filter

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	90	0	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

[illegible]

Linear Filtering

Box Filter

- ★ Replaces each pixel with an average of its neighborhood
- ★ Achieve smoothing effect (remove sharp features)



Linear Filtering

Gaussian Filter or Gaussian Blur

- ★ A Gaussian filter is a linear filter. It's usually used to blur the image or to reduce noise.
- ★ Remove “high-frequency” components from the image (low-pass filter)
- ★ The most general function formula is:

$$h(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{\sigma^2}}$$



Linear Filtering

Bilinear Filtering

- ★ Bilinear interpolation is one of the basic **resampling techniques** in computer vision and image processing.
- ★ It is also called bilinear filtering or bilinear texture mapping.
- ★ Bilinear filtering is a method of texture filtering design to smooth out textures.
- ★ Texture filtering or texture smoothing is the method used to determine the texture color for a texture mapped pixel.

Linear Filtering

Sobel Operator

- ★ It is sometimes called the Sobel–Feldman operator or Sobel filter, is used in image processing and computer vision.
- ★ Sobel operator is used for **edge detection**.
- ★ It is also used to detect two kinds of edges in an image: *Vertical direction* and *Horizontal direction*.

-1	0	1
-2	0	2
-1	0	1

Vertical

1	2	1
0	0	0
-1	-2	-1

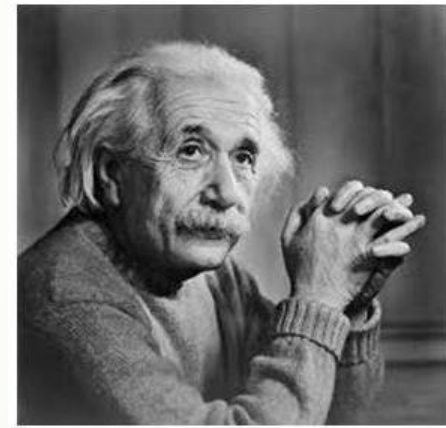
Horizontal

Linear Filtering

Sobel Operator



After applying
vertical mask



After applying
horizontal mask



Linear Filtering

Band-Pass Filter

- ★ A band-pass filter or bandpass filter (BPF) is a device that passes frequencies within a certain range and rejects (attenuates) frequencies outside that range.
- ★ A bandpass attenuates very low and very high frequencies but **retains a middle range band of frequencies**.
- ★ Bandpass filtering can be used to **enhance edges** (suppressing low frequencies) while **reducing the noise** at the same time (attenuating high frequencies).

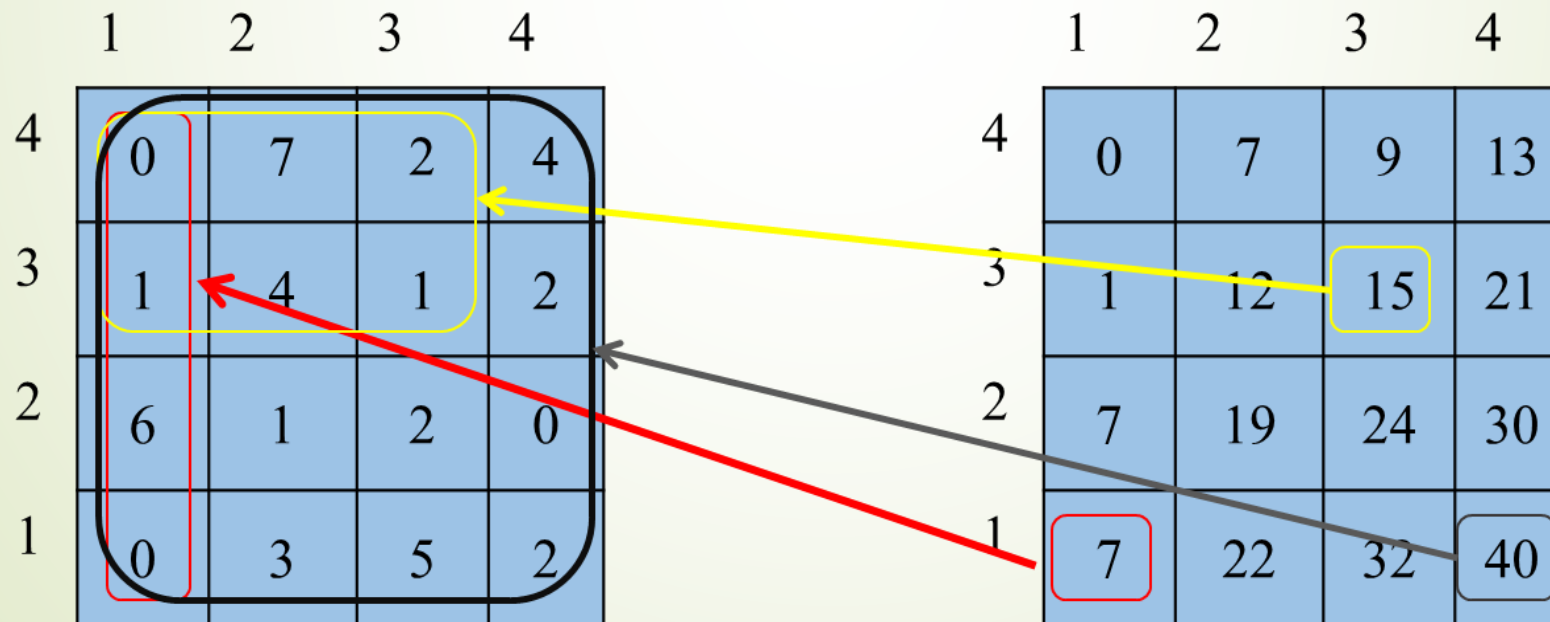
Linear Filtering

Sum Area Table

- ★ It is useful for **averaging pixels** (box filter).
- ★ It is the running sum of all the pixel values from the origin

$$s(i, j) = \sum_{k=0}^i \sum_{l=0}^j f(k, l)$$

- ★ The image $s(i, j)$ is also often called an *integral image*.



Linear Filtering

Sum Area Table

- ★ Disadvantage of summed area tables is that they require $\log M + \log N$ extra bits in the accumulation image compared to the original image . (M and N are the image width and height)
- ★ Extension of summed area tables can also used to approximate other convolution kernels.
- ★ In computer vision, summed area tables have been used in face detection to compute simple multi-scale low-level features.
- ★ Summed area tables could also be used to compute the sums in the sum of squared differences stereo and motion algorithm.

Non-Linear Filtering

- ★ A non-linear filter is one that cannot be done with convolution or Fourier multiplication.
- ★ They can reduce noise levels without blurring edges.
- ★ Non-linear filters are more powerful than linear filter.
 - Median filtering
 - Bilateral filtering
 - Iterated adaptive smoothing and anisotropic diffusion

Non-Linear Filtering

Median Filtering

- ★ The median filter is a non-linear digital filtering technique, often used to remove noise from an image or signal.
- ★ In this filter, we replace the pixel value with the median value.
- ★ Edges are better preserved than with linear filters.
- ★ Best suited for “salt and pepper” noise.

6	2	0
3	97	4
19	3	10

in order:

0, 2, 3, 3, 4, 6, 10, 15, 97

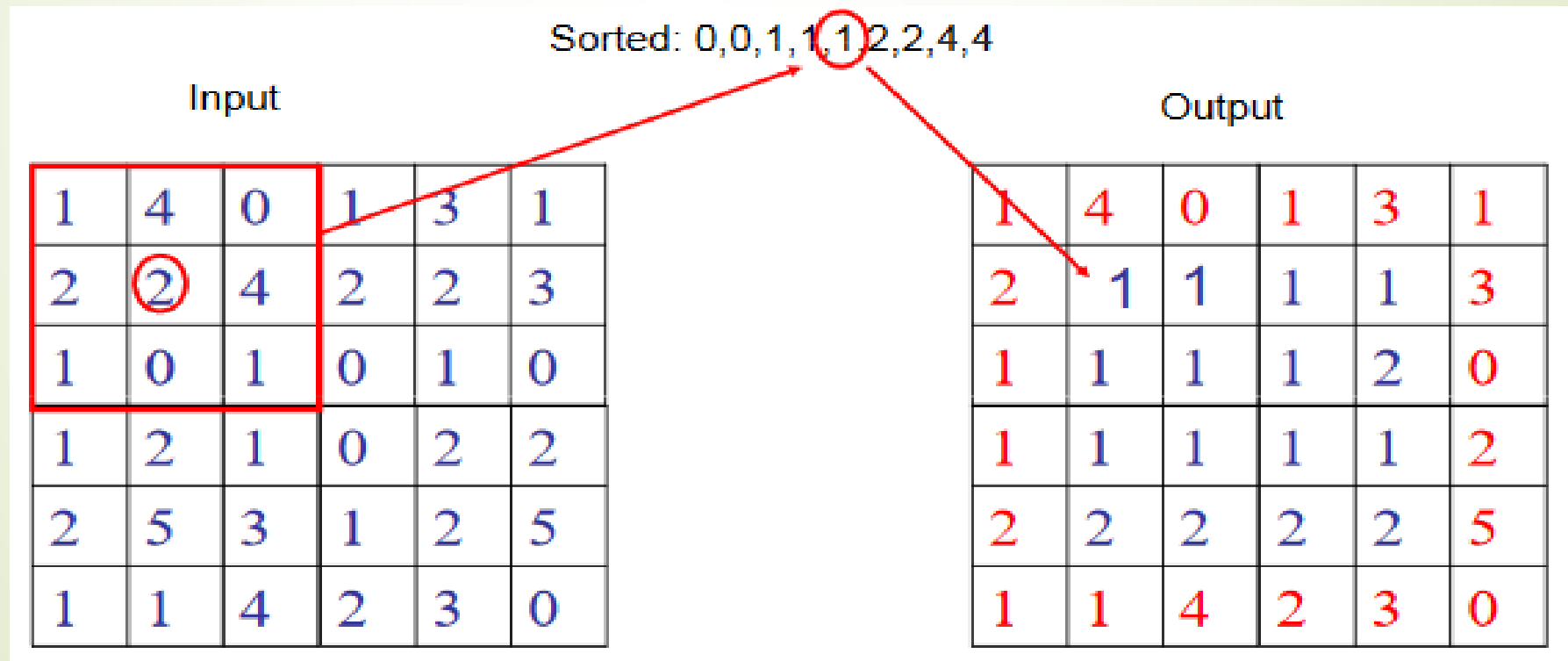
median filtered

*	*	*
*	4	*
*	*	*

Non-Linear Filtering

Median Filtering

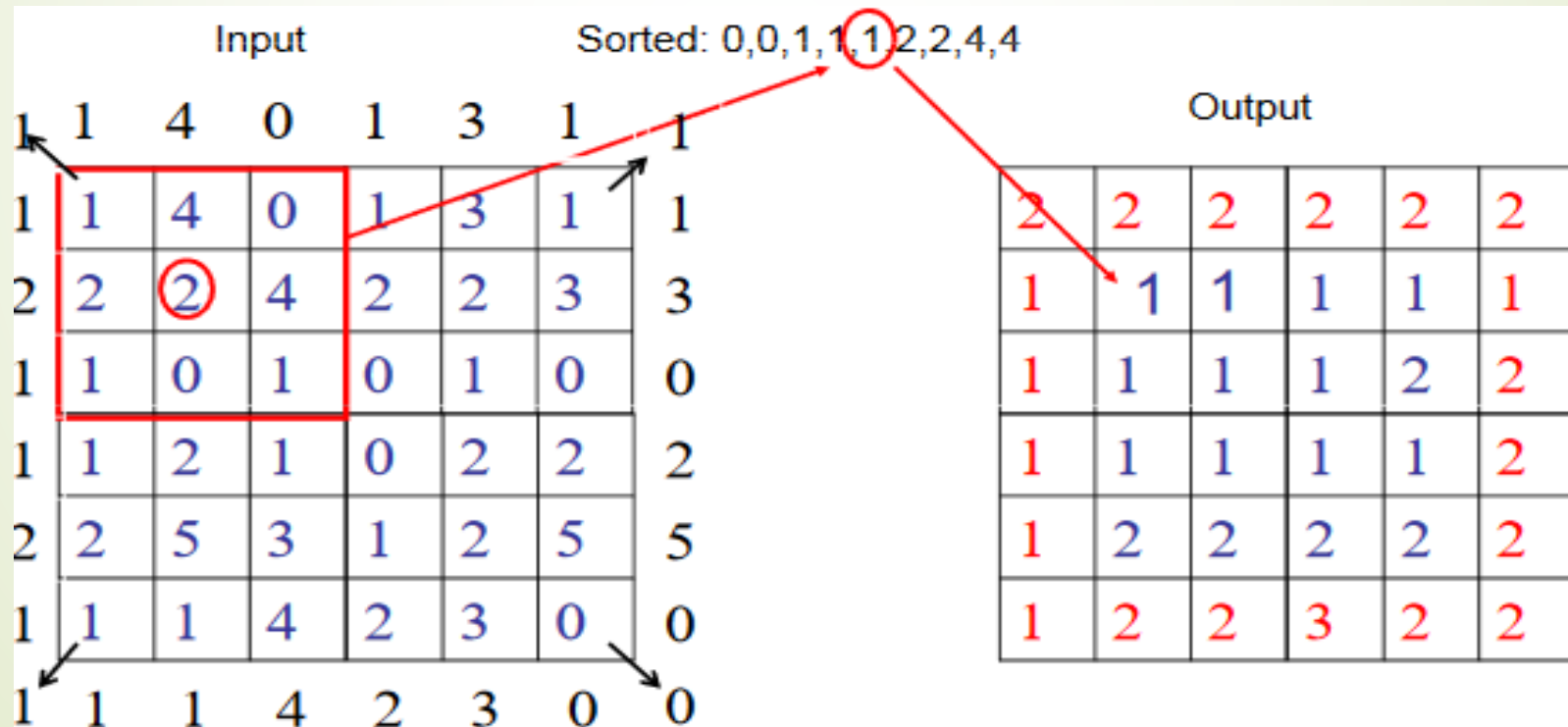
- ★ Example:
- ★ 2D Median filtering example using a 3 x 3 sampling window:
Keeping border values unchanged.



Non-Linear Filtering

Median Filtering

- ★ Example:
- ★ 2D Median filtering example using a 3 x 3 sampling window: Extending border values outside with values at boundary.

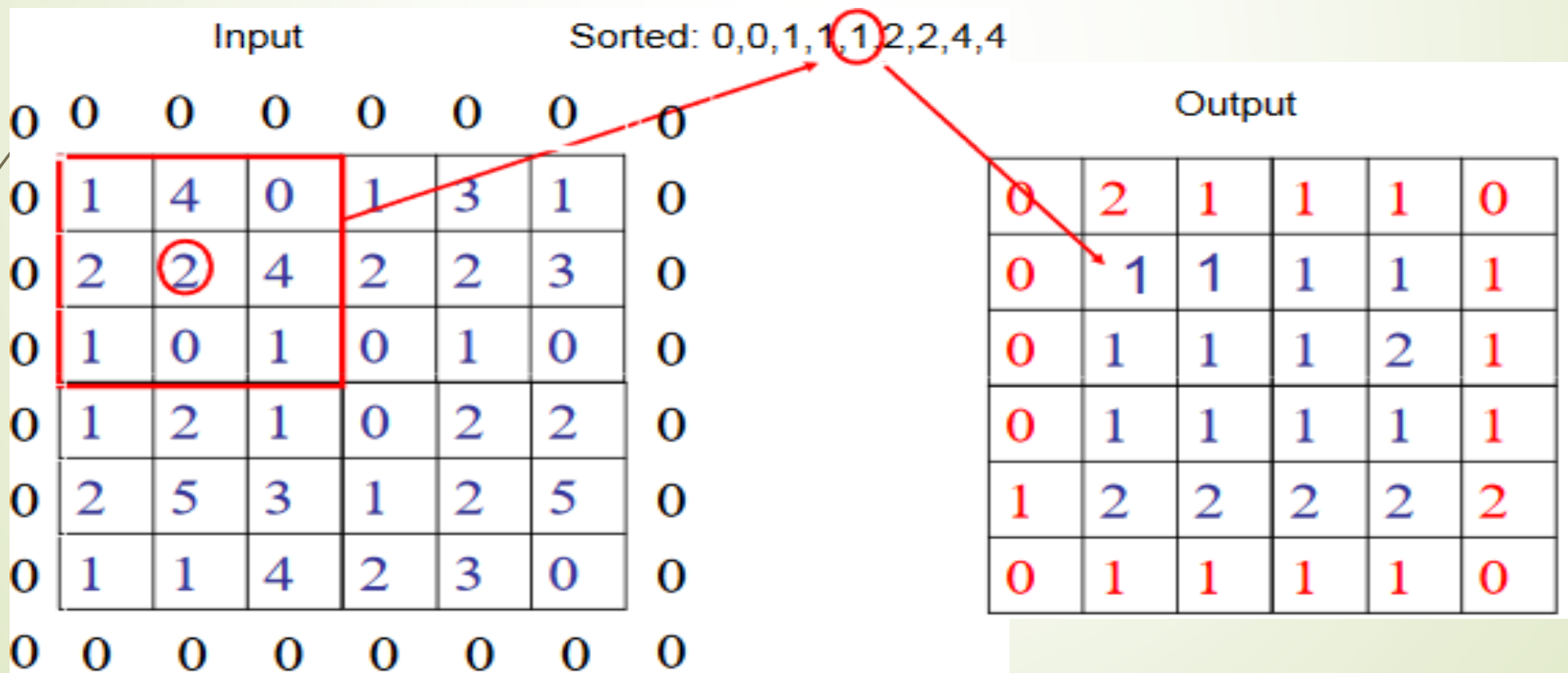


Non-Linear Filtering

Median Filtering

★ Example:

★ 2D Median filtering example using a 3 x 3 sampling window: Extending border values outside with 0s



Non-Linear Filtering

Average or Median Filtering

- ★ Average (or mean) filtering is a method of 'smoothing' images by reducing the amount of intensity variation between neighboring pixels.
- ★ The average filter works by moving through the image pixel by pixel,
- ★ Replacing each value with the average value of neighboring pixels, including itself.
- ★ **Example:** 2D Average filtering example using a 3 x 3 sampling window: Keeping border values unchanged

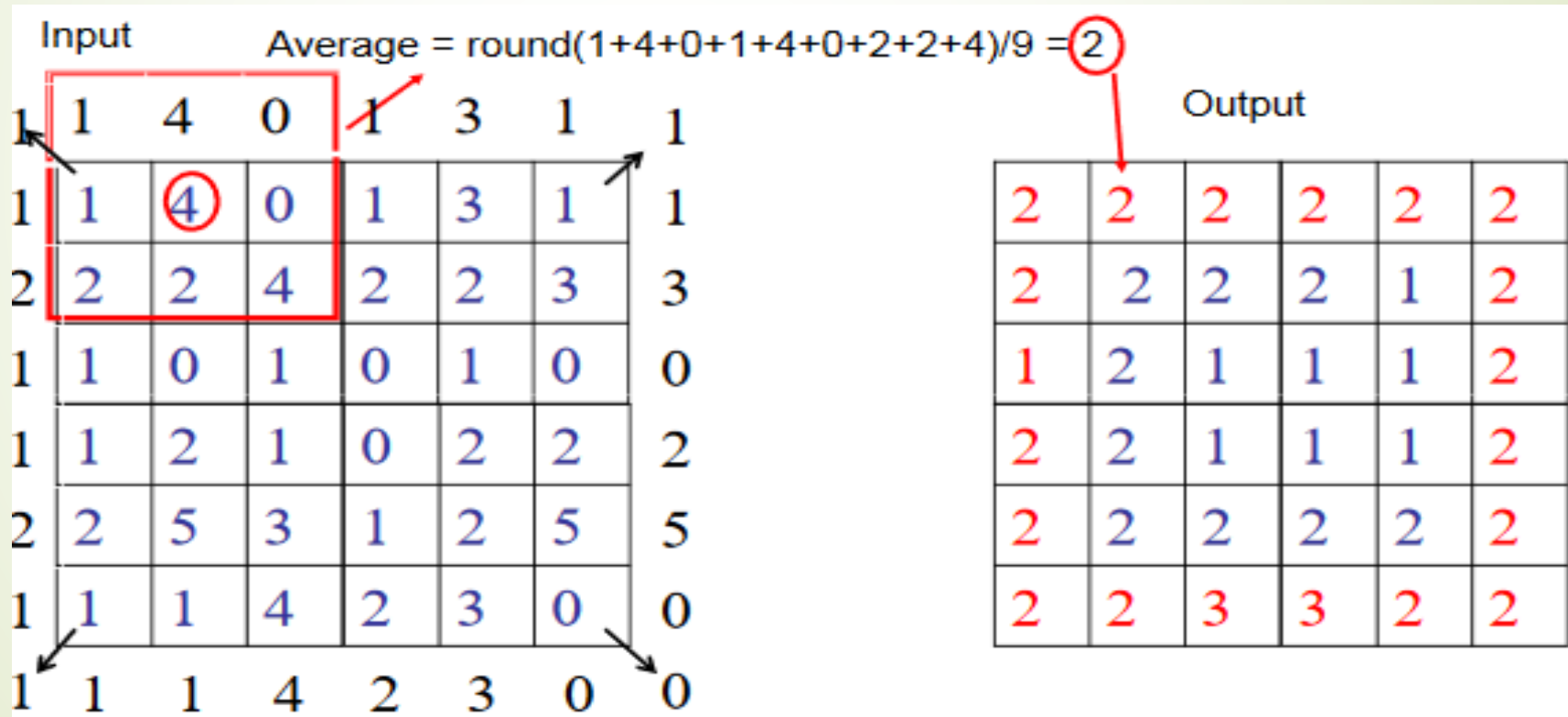
Average = $\text{round}(1+4+0+2+2+4+1+0+1)/9 = 2$

Input						Output		
1	4	0	1	3	1	1	4	0
2	2	4	2	2	3	2	2	2
1	0	1	0	1	0	1	2	1
1	2	1	0	2	2	1	2	1
2	5	3	1	2	5	2	2	2
1	1	4	2	3	0	1	1	4

Non-Linear Filtering

Average or Median Filtering

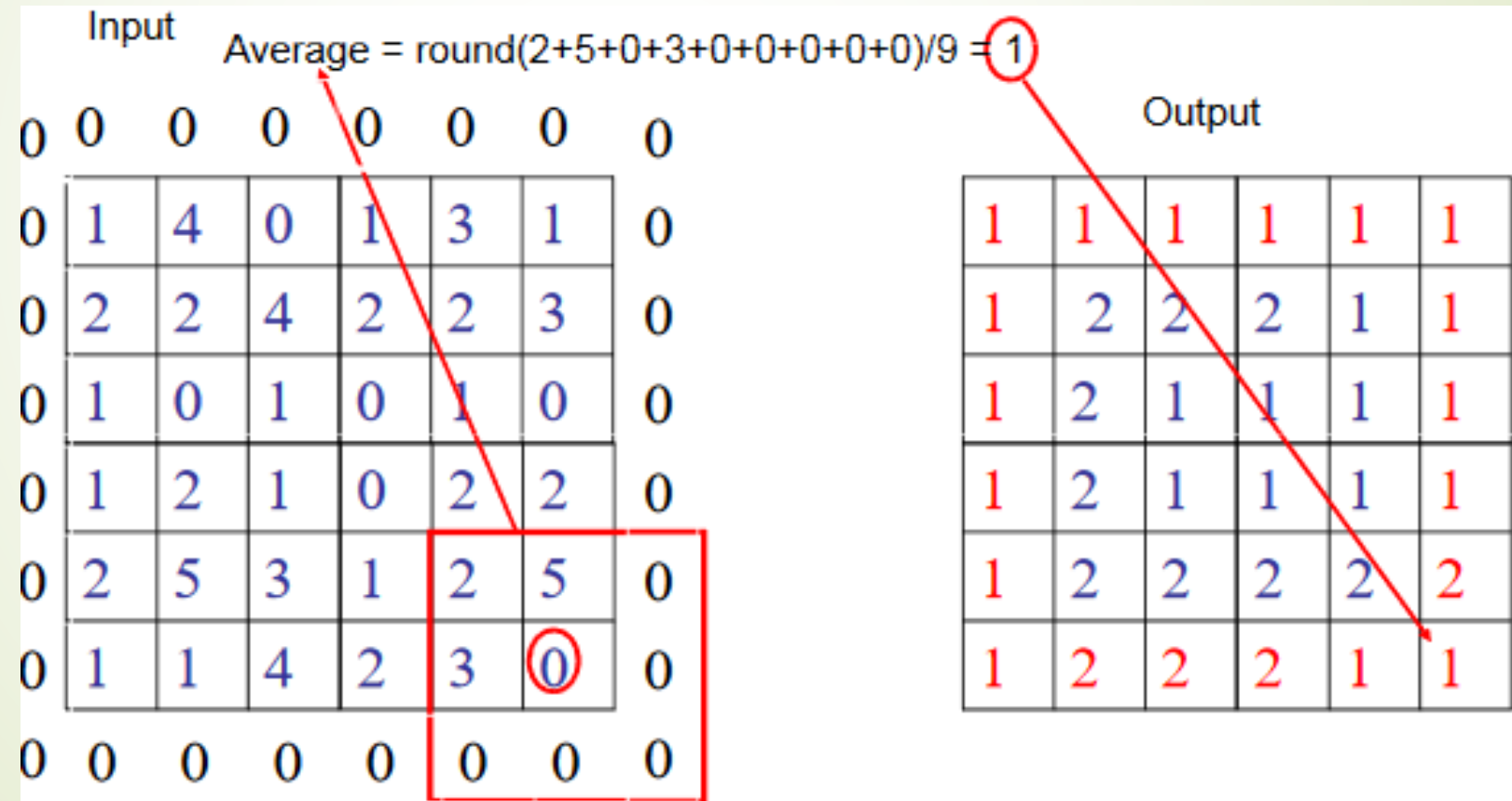
- ★ **Example:** 2D Average filtering example using a 3 x 3 sampling window: Extending border values outside with values at boundary



Non-Linear Filtering

Average or Median Filtering

- ★ **Example:** Median filtering example using a 3 x 3 sampling window: Extending border values outside with 0s (Zero-padding)



Non-Linear Filtering

Bilateral Filtering

- ★ Spatial averaging removes noise and blurs edges
- ★ A bilateral filter is a non-linear, edge-preserving, and noise-reducing smoothing filter for images.
- ★ It replaces the intensity of each pixel with a weighted average of intensity values from nearby pixels.

$$g(i, j) = \frac{\sum_{k,l} f(k, l) w(i, j, k, l)}{\sum_{k,l} w(i, j, k, l)}$$

- ★ Bilateral filtering is quite slow compared to regular separable filtering, use more memory than regular filtering and are not directly applicable to filtering full-color images.

Non-Linear Filtering

Bilateral Filtering

★ Example



- ★ Example of Bilateral filtering
- ★ Low contrast texture has been removed
- ★ Yet edges are well preserved